AI VIET NAM
@aivietnam.edu.vn

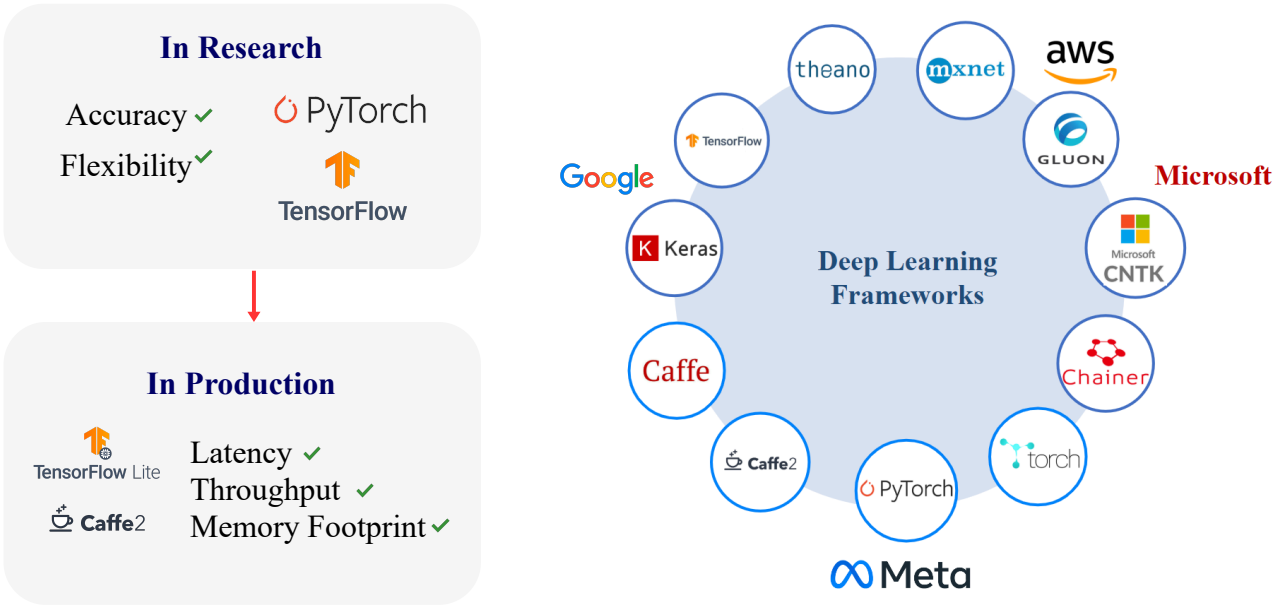
AI VIET NAM – AI COURSE 2025

Tối ưu hiệu suất mô hình với ONNX và TensorRT

Phat-Dat To Nguyen-Thuan Duong Quang-Vinh Dinh

I. Giới thiệu

Sự phát triển vượt bậc và tốc độ lan tỏa nhanh chóng của trí tuệ nhân tạo (AI) trong những năm gần đây đã tạo ra một làn sóng chuyển dịch mạnh mẽ, đưa các mô hình này từ phạm vi nghiên cứu lí thuyết sang ứng dụng vào các vấn đề thực tiễn. Sự chuyển dịch này được thúc đẩy bởi ba yếu tố quan trọng: sự sẵn có của các tập dữ liệu khổng lồ, sự gia tăng năng lực tính toán phần cứng và sự hoàn thiện của các kiến trúc thuật toán tiên tiến. Nhờ đó, các hệ thống AI ngày nay đã có khả năng giải quyết những bài toán phức tạp với độ chính xác cao. Tuy nhiên, chính quy trình chuyển đổi từ thí nghiệm thành một sản phẩm thương mại có khả năng hoạt động ổn định lại bộc lộ những thách thức kĩ thuật liên quan trực tiếp đến vấn đề tương thích môi trường và tối ưu hóa trong vận hành sản phẩm.



Hình 1: Sự khác biệt giữa các yêu cầu trong môi trường nghiên cứu và môi trường triển khai sản phẩm.

Nguyên nhân bắt nguồn từ sự khác nhau giữa các giai đoạn trong vòng đời sản phẩm. Trong giai đoạn nghiên cứu và phát triển, các framework bậc cao như PyTorch hay TensorFlow được ưu tiên sử dụng nhờ khả năng tự động xử lí hầu hết các tác vụ phức tạp liên quan đến phần cứng, giúp các nhà phát triển tập trung cải thiện mô hình mà không cần quan tâm sâu đến việc vận hành của máy tính. Tuy nhiên, chính việc phụ thuộc vào các công cụ hỗ trợ sẵn có cũng khiến mô hình bị ràng buộc vào hệ sinh thái cố định. Các chi phí triển khai, vận hành mô hình cũng thường bị bỏ qua thay vì xem xét đến.

Ngược lại, môi trường vận hành lại chịu sự chi phối nghiêm ngặt của các yêu cầu như độ trễ phản hồi theo thời gian thực, thông lượng xử lí phải lớn và hiệu quả tối ưu trong tiêu thụ năng lượng. Khi một mô hình được đưa vào môi trường sản xuất quy mô lớn, tính linh hoạt của giai đoạn phát triển lại trở thành gánh nặng về hiệu năng, gây lãng phí tài nguyên và làm chậm tốc độ xử lí. Lúc này, thách thức đặt ra không còn ở việc phải thay đổi về mã nguồn, mà còn phải giải quyết mâu thuẫn giữa nhu cầu thay đổi liên tục trong thử nghiệm và yêu cầu cố định, tối ưu hóa triệt để trong vận hành. Chính rào cản này đã ngăn cản việc chuyển giao hiệu quả các thành tựu nghiên cứu tiên tiến vào các ứng dụng thực tế.

Để giải quyết vấn đề này, Open Neural Network Exchange (ONNX) được giới thiệu như một cầu nối giúp chuyển giao mô hình từ môi trường phát triển sang môi trường vận hành một cách nguyên vẹn. ONNX cho phép các mô hình được phát triển trên đa dạng các framework khác nhau đều có thể được đọc và thực thi bởi bất kì hệ thống nào hỗ trợ chuẩn này. Thông qua việc chuẩn hóa giao tiếp giữa các nền tảng, ONNX đã biến các mô hình thuật toán phức tạp thành các thành phần phần mềm độc lập, sẵn sàng để tích hợp vào bất kỳ hệ thống sản xuất nào.
Tuy nhiên, tính di động và khả năng tương tác của ONNX mới chỉ là điều kiện cần nhưng chưa đủ cho các ứng dụng yêu cầu hiệu năng cao. Để khai thác triệt để sức mạnh tính toán của các phần cứng hiện đại, cần có một công cụ chuyên biệt có khả năng biên dịch lại biểu diễn trung gian này thành mã máy tối ưu. Đây chính là vai trò của NVIDIA TensorRT trong quy trình triển khai. TensorRT là một bộ công cụ bao gồm trình tối ưu hóa và môi trường thực thi, được thiết kế đặc biệt cho kiến trúc phần cứng của NVIDIA. Sự kết hợp giữa ONNX và TensorRT tạo nên một quy trình làm việc liền mạch và hiệu quả cao. Sau khi mô hình được chuẩn hóa về dạng ONNX, TensorRT sẽ tiếp nhận biểu diễn này và thực hiện các kĩ thuật tối ưu hóa để đạt được hiệu suất suy luận cực đại.

Bài viết này sẽ đi sâu phân tích cơ chế kĩ thuật của quy trình chuyển đổi và tối ưu hóa này, làm rõ cách thức hai công nghệ này phối hợp để giải quyết triệt để bài toán triển khai mô hình trong thực tế.

Mục lục

I. Giới thiệu **1**

II. Section 1: Motivation **4**

II.1. Bài toán chuyển đổi và tương thích mô hình 4

II.2. Một vài metrics đánh giá 4

III. Section 2: ONNX **6**

III.1. Open Neural Network Exchange 6

III.2. ONNX Runtime 11

III.3. Thực hành ONNX 15

IV. Section 3: TensorRT **19**

IV.1. Tổng quan về TensorRT 19

IV.2. Các kĩ thuật tối ưu hóa trong TensorRT 22

IV.3. trtexec 26

IV.4. Dynamic Shape 27

V. Thực hành ONNX và TensorRT với YOLO26 . . . . . **29**

V.1. Set up 29

V.2. Chuyển đổi mô hình YOLO26 sang ONNX và TensorRT 30

V.3. Xây dựng ứng dụng API và Inference 33

VI. Câu hỏi ôn tập **39**

VII. Tài liệu tham khảo **42**

II. Section 1: Motivation

II.1. Bài toán chuyển đổi và tương thích mô hình

Một trong những trở ngại lớn nhất trong việc triển khai các hệ thống trí tuệ nhân tạo hiện nay là sự thiếu đồng bộ giữa các công cụ phát triển. Khi mà các thư viện học máy phổ biến khác nhau đều vận hành theo những quy tắc thiết kế riêng biệt về cấu trúc dữ liệu và toán tử. Sự khác biệt này khiến các mô hình sau khi huấn luyện bị phụ thuộc hoàn toàn vào thư viện gốc, gây khó khăn khi cần trích xuất và vận hành trên các môi trường khác nhau.

Trong môi trường nghiên cứu, các framework bậc cao được tối ưu hóa cho năng suất của con người. Các nền tảng này cung cấp cơ chế thực thi linh hoạt và đồ thị động giúp các nhà phát triển dễ dàng gỡ lỗi và thay đổi kiến trúc mạng tức thì. Tuy nhiên, chính sự linh hoạt này lại đi kèm với chi phí lớn về tài nguyên do phải duy trì các cấu trúc hỗ trợ huấn luyện phức tạp như lưu trữ gradient hay đồ thị đạo hàm. Ngược lại, môi trường sản xuất lại đặt trọng tâm vào hiệu suất của máy móc, nơi các yêu cầu về độ trễ thấp, thông lượng cao và tối ưu hóa chi phí phần cứng là trên hết. Việc đưa một môi trường phát triển cồng kềnh vào sản phẩm thực tế sẽ tạo ra sự dư thừa, gây lãng phí tài nguyên tính toán và khó đảm bảo tính ổn định. Vì thế, việc chuyển đổi mã nguồn giữa các môi trường khác nhau là cần thiết, nhưng thường gặp nhiều rủi ro về lỗi tương thích nghiêm trọng nếu can thiệp trực tiếp vào mã nguồn.

Vấn đề triển khai càng trở nên khó khăn hơn khi xét đến sự đa dạng của phần cứng ngày nay. Mỗi loại vi xử lý như CPU, GPU hay NPU đều yêu cầu những tập lệnh tối ưu riêng. Một mô hình được phát triển bằng các framework thân thiện với GPU chưa chắc đã chạy tốt được trên các thiết bị chỉ có CPU hay NPU. Lúc này các kĩ sư phải đối mặt với bài toán chuyển đổi ngôn ngữ hay tập lệnh để tương thích với thiết bị đích. Việc phải phát triển và duy trì hàng loạt công cụ chuyển đổi thủ công gây lãng phí nguồn lực khổng lồ.

Để giải quyết những vấn đề nêu trên, các nhà phát triển đặt ra yêu cầu về một cơ chế trung gian thống nhất có khả năng chuẩn hóa toàn bộ quá trình chuyển đổi từ phát triển lên môi trường sản phẩm, cho phép tách biệt hoàn toàn tầng thiết kế thuật toán khỏi tầng thực thi phần cứng. Khi đó, quá trình tích hợp không còn là những nỗ lực để kết nối các công cụ riêng lẻ, mà trở thành một quy trình có tính chuẩn hóa, nơi mọi nền tảng phát triển và thiết bị vận hành đều giao tiếp qua một ngôn ngữ duy nhất.

II.2. Một vài metrics đánh giá

Trong triển khai thực tế, việc đánh giá hiệu quả của mô hình không còn chỉ dựa trên độ chính xác mà phụ thuộc lớn vào các chỉ số hiệu năng phần cứng. Để đo lường sự tối ưu của các công cụ như ONNX hay TensorRT, ta thường sử dụng ba thước đo định lượng chính, đó là độ trễ, thông lượng và mức tiêu thụ bộ nhớ.

II.2.1. Độ trễ (Latency)

Độ trễ là khoảng thời gian trôi qua kể từ khi hệ thống tiếp nhận dữ liệu đầu vào đến khi trả ra kết quả dự đoán. Đây là chỉ số quan trọng nhất đối với các ứng dụng real-time như xe tự hành hay nhận diện khuôn mặt, nơi phản hồi tức thì là yêu cầu bắt buộc.

Độ trễ (L) thường được đo bằng đơn vị mili giây (ms) và được tính theo công thức:

$$L = t_{end} - t_{start},$$

trong đó t_{start} là thời điểm request được gửi vào mô hình và t_{end} là thời điểm kết quả hoàn tất và sẵn sàng trả về.

Trong thực tế, thay vì chỉ quan tâm đến độ trễ trung bình (L_{avg}), ta thường tập trung vào chỉ số Percentile Latency như P95 hoặc P99. Ví dụ chỉ số P99 = 20ms nghĩa là 99% các yêu cầu được xử lí rất nhanh, ít hơn 20ms. Điều này cho ta biết rằng trải nghiệm người dùng sẽ ổn định ngay cả khi hệ thống được truy cập bởi rất nhiều người dùng cùng lúc.

II.2.2. Thông lượng (Throughput)

Thông lượng đo lường khả năng xử lí một khối lượng công việc của hệ thống trong một đơn vị thời gian. Chỉ số này đặc biệt quan trọng đối với các hệ thống ngoại tuyến hoặc các trung tâm dữ liệu, nơi mục tiêu là xử lí được càng nhiều dữ liệu càng tốt.

Thông lượng (T) thường được đo bằng đơn vị truy vấn trên giây (Queries Per Second - QPS) hoặc số ảnh trên giây (Images Per Second - IPS), được tính theo công thức:

$$T = \frac{N}{L_{batch}},$$

với N là số lượng mẫu (batch size) được xử lý đồng thời trong một lần chạy và L_{batch} là thời gian xử lí batch dữ liệu tương ứng.

Thường có sự đánh đổi giữa latency và throughput. Tăng batch size thường làm tăng throughput do tận dụng tốt khả năng song song của GPU nhưng đồng thời cũng làm tăng latency.

II.2.3. Memory Footprint

Memory Footprint hay mức tiêu thụ VRAM/RAM đo lường lượng tài nguyên bộ nhớ mà mô hình chiếm dụng trong quá trình khởi chạy và thực thi. Chỉ số này cực kì quan trọng khi triển khai trên các thiết bị nhúng hoặc thiết bị biên có tài nguyên hạn chế.

Tổng bộ nhớ tiêu thụ (M_{total}) được ước tính bởi tổng của các thành phần:

$$M_{total} = M_{weights} + M_{activations} + M_{workspace},$$

trong đó $M_{weights}$ là bộ nhớ tĩnh dùng để lưu trữ các tham số của mô hình, $M_{activations}$ là bộ nhớ động dùng để lưu trữ các giá trị trung gian giữa các lớp trong quá trình tính toán và $M_{workspace}$ là vùng nhớ tạm thời mà các thuật toán yêu cầu để thực hiện phép tính. Đơn vị đo lường thường là Megabyte (MB) hoặc Gigabyte (GB).

III. Section 2: ONNX

III.1. Open Neural Network Exchange

Open Neural Network Exchange (ONNX) là công cụ mã nguồn mở cung cấp định dạng chuẩn để biểu diễn chung cho các mô hình học máy khác nhau. ONNX ban đầu được phát triển bởi Facebook và Microsoft, hiện nay ONNX đã nhận được sự hỗ trợ rộng rãi từ hàng loạt các tổ chức công nghệ hàng đầu, bao gồm IBM, Amazon và Google.

Thay vì đóng vai trò là công cụ chuyển đổi mã nguồn giữa các thư viện, ONNX cung cấp một định dạng biểu diễn mô hình thống nhất. Cơ chế này giúp tách biệt hoàn toàn giai đoạn phát triển khỏi giai đoạn vận hành. Các kỹ sư có thể tự do lựa chọn công cụ tốt nhất để huấn luyện mô hình, sau đó đóng gói chúng dưới dạng ONNX để triển khai trên các hệ thống hiệu năng cao như TensorRT hay ONNX Runtime mà không bị ràng buộc bởi các thư viện gốc ban đầu.



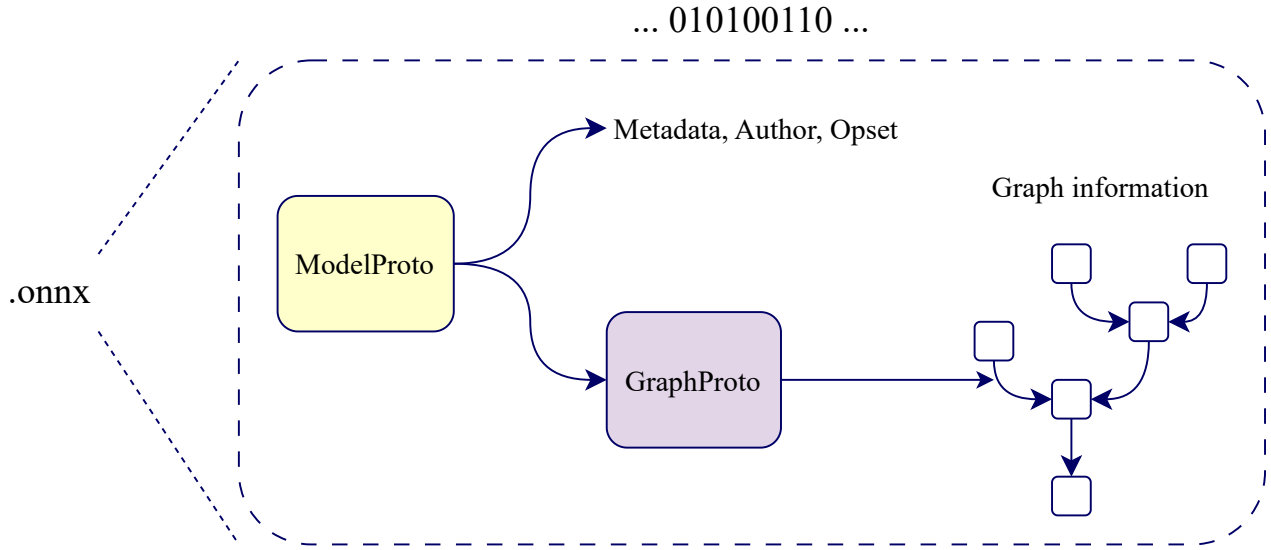
Hình 2: ONNX chuyển đổi các định dạng khác nhau về chuẩn chung .onnx, sau đó deploy lên các phần cứng khác nhau.

Khi một mô hình được chuyển đổi sang chuẩn ONNX, sự phụ thuộc vào các thư viện phát triển hay mã nguồn ban đầu bị loại bỏ hoàn toàn. Mô hình lúc này sẽ trở thành một cấu trúc độc lập nhưng vẫn giữ trọn vẹn thông tin về kiến trúc mạng và giá trị tham số. Tính chất độc lập này cho phép vận hành mô hình trên bất kỳ hệ thống nào hỗ trợ chuẩn ONNX mà không yêu cầu cài đặt lại môi trường huấn luyện phức tạp.

Ngoài ra, để khai thác triệt để sức mạnh phần cứng, ONNX sử dụng cơ chế tính toán dạng đồ thị tĩnh. Khác với các thư viện sử dụng cơ chế đồ thị động để chuyên sâu vào huấn luyện mô hình như PyTorch hay TensorFlow, cấu trúc tĩnh giúp hệ thống có thể quan sát trước toàn bộ các luồng xử lí dữ liệu, từ đó cho phép trình biên dịch thực hiện các kĩ thuật tối ưu hóa hiệu quả. Ngoài ra, chuẩn ONNX có thể chạy trên nhiều loại thiết bị đích khác nhau, giúp giải phóng các nhà sản xuất phần cứng khỏi áp lực phải chạy đua hỗ trợ các thư viện phần mềm, cho phép họ tập trung nguồn lực để xây dựng một trình biên dịch tối ưu duy nhất dựa trên chuẩn chung ONNX. Kết quả là ONNX không chỉ kết nối các nền tảng lập trình mà còn thống nhất được sự phân mảnh của thế giới phần cứng, đảm bảo tính toàn vẹn của giải pháp AI khi triển khai trên quy mô lớn.

III.1.1. Kiến trúc tệp tin.

Mô hình được chuyển sang định dạng ONNX sẽ được lưu vào một tệp có phần mở rộng là .onnx.

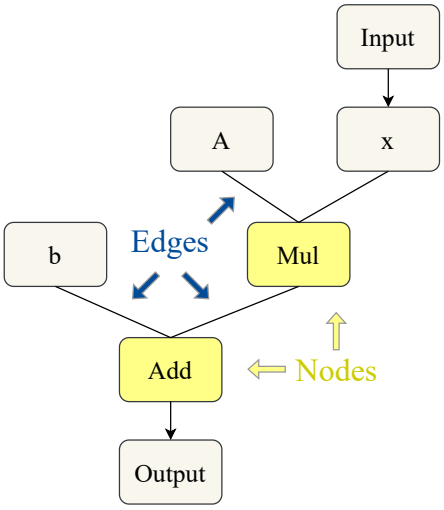


Hình 3: Kiến trúc bên trong tệp tin .onnx

Để đóng gói mô hình vào một tệp tin trung gian, ONNX sử dụng giao thức Protocol Buffers (Protobuf). Thay vì lưu trữ thông tin mô hình dưới dạng văn bản vốn tốn nhiều không gian vùng nhớ, Protobuf lưu trữ tập tin dưới dạng nhị phân, trong đó các trọng số và cấu trúc mạng được nén lại thành các dãy bit liên tục và có cấu trúc chặt chẽ bằng cách định danh cụ thể cho các thành phần trong dữ liệu. Cách lưu giữ này giúp loại bỏ chi phí để đọc hiểu văn bản, giảm đáng kể kích thước tệp tin và tăng tốc độ nạp mô hình vào bộ nhớ mà vẫn giữ toàn vẹn được cấu trúc của mô hình. Định dạng nhị phân này còn cho phép các hệ thống viết bằng ngôn ngữ cấp thấp như C hay Java, hoặc các thiết bị nhúng có thể sử dụng mô hình ngay lập tức mà không cần cài đặt môi trường phụ thuộc mã nguồn gốc ban đầu.

Dữ liệu trong tệp tin nhị phân được tổ chức theo cấu trúc phân cấp. Bao bọc bên ngoài là ModelProto. Đây là cấu trúc dữ liệu tổng quát chứa các metadata cần thiết để định danh mô hình, bao gồm thông tin phiên bản, tác giả và các tập hợp toán tử (Opset). Ngoài ra ModelProto còn chứa GraphProto, hay đồ thị tính toán. Thành phần này định nghĩa toàn bộ luồng xử lí dữ liệu của mô hình. GraphProto chứa các khai báo về input và output, xác định rõ kích thước của các tensor và kiểu dữ liệu cho từng tham số.

Bên trong GraphProto, các thành phần được tổ chức chặt chẽ theo mô hình lí thuyết đồ thị, bao gồm tập hợp các đỉnh và cạnh. Các đỉnh (node) đóng vai trò là các đơn vị xử lí trung tâm. Mỗi đỉnh tương ứng với một phép toán cụ thể như Convolution, Add, hay ReLU và chịu trách nhiệm biến đổi dữ liệu. Các đỉnh được liên kết thông qua các cạnh (edge), đại diện cho các tensor dữ liệu. Trong đồ thị này, các cạnh không được định nghĩa riêng biệt mà được tạo ra thông qua cơ chế định danh tham số. Đầu ra của một đỉnh sẽ trở thành cạnh kết nối khi nó được sử dụng làm đầu vào cho một đỉnh khác. Cấu trúc này tạo thành một đồ thị có hướng không chu trình (Directed Acyclic Graph - DAG), đảm bảo dữ liệu luôn di chuyển theo một chiều duy nhất từ lớp đầu vào đến lớp đầu ra.



Hình 4: Minh họa đồ thị tính toán.

Để tinh chỉnh thông số của các đỉnh trong đồ thị mà không làm thay đổi cấu trúc, ONNX sử dụng các thuộc tính (attributes), được gán riêng cho mỗi đỉnh. Khác với các cạnh mang dữ liệu động thay đổi liên tục trong quá trình tính toán, thuộc tính là các giá trị cấu hình tĩnh gắn liền với từng đỉnh cụ thể. Ví dụ, trong một đỉnh thực hiện phép tích chập, các thuộc tính sẽ quy định kernel size, stride hay padding. Việc tách biệt này giúp hệ thống xác định rõ ràng giữa luồng thông tin cần xử lý và logic vận hành của thuật toán.

Cuối cùng, để đồ thị có thể thực hiện các phép tính có ý nghĩa, nó cần được cung cấp các giá trị tham số đã được huấn luyện trước đó, quá trình này được thực hiển bởi trình một trình khởi tạo gọi là TensorProto. ONNX sử dụng TensorProto để thực hiện gán các giá trị trọng số và bias vào một cạnh được định danh cụ thể. Lúc này toàn bộ tri thức của mô hình sẽ được đưa vào cấu trúc đồ thị một cách tự động và chính xác ngay trước khi quá trình thực thi bắt đầu.

III.1.2. Kiểu dữ liệu hỗ trợ, toán tử và Opset.

Mỗi ngôn ngữ lập trình đều có cách định nghĩa và lưu trữ kiểu dữ liệu riêng biệt, điều này dễ dẫn đến sự sai lệch giá trị khi chuyển đổi mô hình từ môi trường này sang môi trường khác. Để khắc phục sự bất đồng này, ONNX tạo ra một bộ quy tắc định dạng dữ liệu chung, bắt buộc mọi ngôn ngữ lập trình khi kết nối với ONNX đều phải chuyển đổi dữ liệu của mình về các định dạng chuẩn của ONNX. Các kiểu dữ liệu bao gồm các định dạng số học phổ biến như số thực: float16, float32, float64 hay các kiểu số nguyên: int8, int16, int32, int64. Bên cạnh đó, để phục vụ các thuật toán học máy khác nhau, ONNX còn hỗ trợ các kiểu dữ liệu non-tensor như chuỗi ký tự (Sequence) và cấu trúc ánh xạ (Map).

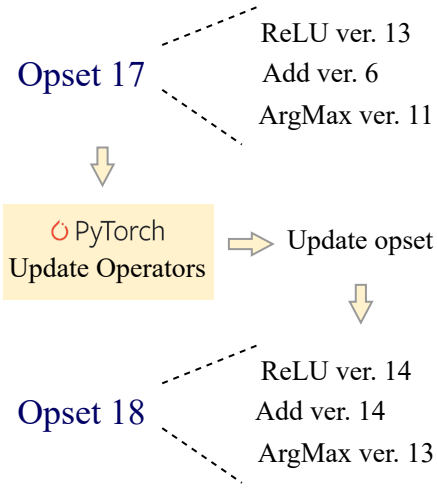
Ngoài ra, ONNX cũng cung cấp một thư viện các toán tử (Operator) được tổ chức theo các miền phân loại khác nhau. Tất cả các hàm tính toán từ mã nguồn gốc sẽ được chuyển đổi thành các operator tương ứng được ONNX hỗ trợ. Miền quan trọng nhất là ai.onnx, chứa toàn bộ các toán tử Deep Learning hiện đại và ai.onnx.ml dành cho các thuật toán học máy cổ điển. Ngoài ra, ONNX cũng cho phép khả năng chuyển đổi linh hoạt thông qua cơ chế phân rã (Decomposition). Khi gặp các hàm phức tạp từ framework nguồn mà không có toán tử hỗ trợ tương ứng, ONNX

cho phép phân rã hàm đó thành một chuỗi các toán tử nguyên thủy. Ví dụ, một hàm kích hoạt phức tạp như GELU có thể được biểu diễn thành tổ hợp của các phép toán cơ bản Div, Erf, Add, Mul, đảm bảo mọi kiến trúc mô hình đều có thể được chuyển đổi thành công.

Để quản sự thay đổi, ONNX gán cho mỗi toán tử một Operator Version (Opver). Khi một toán tử cần bổ sung tính năng mới hoặc thay đổi công thức tính toán để tăng độ chính xác, Opver riêng toán tử đó sẽ được nâng lên. Ví dụ, với toán tử Resize:

- **Operator Version 10**: Toán tử này chỉ chấp nhận đầu vào là tỉ lệ phóng to/thu nhỏ.
- **Operator Version 11**: Toán tử được cập nhật lớn, yêu cầu thêm các tham số như ROI để định nghĩa cách tính tọa độ điểm ảnh.

Định nghĩa của một toán tử ở hai phiên bản riêng biệt là hoàn toàn khác nhau.

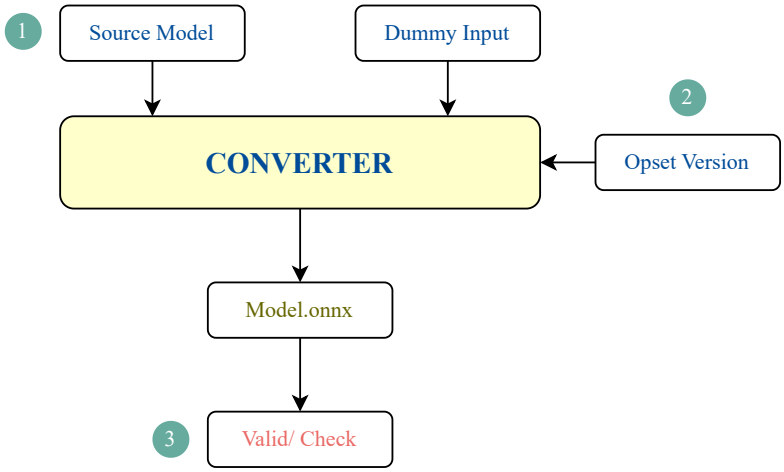


Hình 5: Opset trong ONNX.

Tuy nhiên, trong một đồ thị chứa hàng nghìn toán tử khác nhau, việc quản lý phiên bản riêng lẻ cho từng node là bất khả thi. Do đó, ONNX sử dụng khái niệm Operator Set (Opset) để quản lí một cách đồng bộ. Opset quy định một tập hợp các các phiên bản toán tử cụ thể được phép sử dụng cùng nhau tại một thời điểm. Khi xuất mô hình với khai báo Opset_Version, hệ thống sẽ tự động ánh xạ đến tập các phiên bản toán tử tương ứng được quy định sẵn. Cơ chế này giúp đảm bảo tính tương thích và thực thi nhanh chóng do trình thực thi chỉ cần nhìn vào số hiệu Opset của tệp tin là có thể biết chính xác cần dùng phiên bản nào cho từng phép toán trong hàng nghìn phép toán của mô hình.

III.1.3. Quy trình chuyển đổi mô hình trong ONNX

Quy trình chuyển đổi mô hình sang định dạng ONNX được thực hiện theo ba bước:



Hình 6: Các bước chuyển đổi mô hình trong ONNX.

Bước 1. Khởi tạo và thực hiện truy vết
Đối với các framework sử dụng cơ chế đồ thị động, cấu trúc mạng không tồn tại sẵn dưới dạng cố định mà được hình thành ngay trong quá trình thực thi. Để chuyển sang đồ thị tĩnh, ONNX cần biết được toàn bộ kiến trúc mạng này bằng cách sử dụng kĩ thuật truy vết (Tracing). ONNX yêu cầu cung cấp một đầu vào giả (Dummy Input) là một tensor có cùng kích thước và kiểu dữ liệu với dữ liệu thực tế.

Quá trình sau đó diễn ra như sau:

- Hệ thống thực hiện một lượt truyền Dummy Input đi qua mô hình. Trong quá trình này, một bộ chuyển đổi (Converter) sẽ ghi lại trình tự các phép toán mà dữ liệu chạm vào.
- Kết quả là một bản đồ truy vết về các logic thực thi được tạo ra. Cơ chế này còn giúp loại bỏ các mã lệnh thừa thãi không tham gia vào quá trình tính toán, thực hiện một bước làm sạch cơ bản ngay tại thời điểm xuất file.

Bước 2. Khai báo Opset và ánh xạ các toán tử
Sau khi có được truy vết của các bước thực thi, Converter thực hiện ánh xạ các hàm của framework nguồn sang các toán tử chuẩn của ONNX. Quá trình này chịu sự chi phối nghiêm ngặt của Opset. Opset sẽ quy định danh sách các toán tử hợp lệ được phép sử dụng trong tệp đầu ra. Nếu một hàm phức tạp từ nguồn không có toán tử tương ứng trong Opset, Converter sẽ kích hoạt cơ chế phân rã toán tử.

Bước 3. Đóng gói và kiểm tra
Kết quả cuối cùng là một tệp .onnx chứa đồ thị tính toán tĩnh. Trước khi đưa vào sử dụng, tệp tin này cần trải qua bước kiểm tra để xác minh tính toàn vẹn của cấu trúc đồ thị và chính xác về chiều giữa các tensor đầu vào/đầu ra. Tại giai đoạn này, tệp ONNX mới chỉ đảm bảo tính chính xác về mặt logic và cấu trúc dữ liệu. Để đạt được hiệu suất xử lí cao nhất trên các phần cứng cụ thể, mô hình cần tiếp tục trải qua các kĩ thuật tối ưu hóa chuyên sâu khi được nạp vào môi trường thực thi (Runtime) như ONNX Runtime hay TensorRT, nội dung sẽ được thảo luận chi tiết trong các phần tiếp theo.

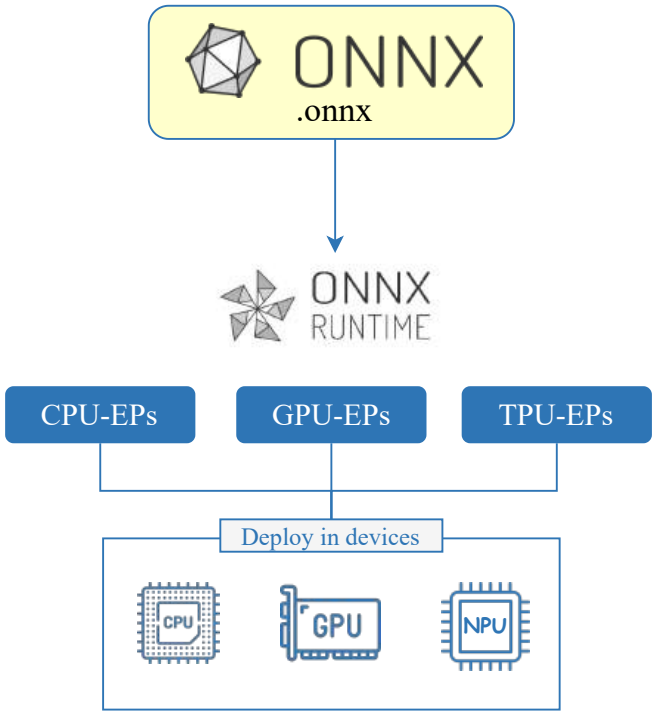
Để hỗ trợ hệ sinh thái đa dạng của các thư viện học máy, ONNX cung cấp các Converters chuyên biệt giúp kết nối giữa framework nguồn và định dạng đích. Dựa trên các nền tảng phổ biến như PyTorch, TensorFlow, Keras hay Scikit-learn, chúng ta có các công cụ chuyển đổi tiêu chuẩn sau:

- **PyTorch (torch.onnx):** Được tích hợp sẵn trong thư viện PyTorch. Người dùng chỉ cần gọi hàm torch.onnx.export() cùng với một dummy_input để xuất mô hình trực tiếp mà không cần cài đặt thêm phần mềm bên ngoài.
- **TensorFlow/Keras (tf2onnx):** Là thư viện độc lập dùng để chuyển đổi các định dạng SavedMode, Keras model (.h5) hoặc GraphDef sang ONNX. Công cụ này đặc biệt hữu ích trong việc chuẩn hóa các phiên bản TensorFlow cũ về chuẩn chung của ONNX.
- **Scikit-learn (sklearn-onnx):** Dành riêng cho các thuật toán học máy cổ điển. Thư viện này chuyển đổi mô hình sang các toán tử thuộc miền ai.onnx.ml, cho phép triển khai các mô hình thống kê nhẹ nhàng trên nhiều nền tảng mà không cần GPU.

III.2. ONNX Runtime

III.2.1. Tổng quan

ONNX Runtime (ORT) là một High-performance Inference Engine (tạm dịch: trình suy luận hiệu năng cao) được thiết kế chuyên biệt để vận hành các mô hình AI trong môi trường thực tế. ONNX Runtime hoạt động như một cầu nối kĩ thuật, nhận đầu vào là các mô hình định dạng chuẩn ONNX và thực thi chúng trên hạ tầng phần cứng vật lí. Mục tiêu chính của ORT là tối ưu hóa tốc độ xử lí và đảm bảo tính tương thích trên đa nền tảng.



Hình 7: ONNX Runtime được xây dựng dựa trên cấu trúc phân tầng. Nhận đầu vào từ ONNX, thông qua các Execution Provider để triển khai trên phần cứng cụ thể.

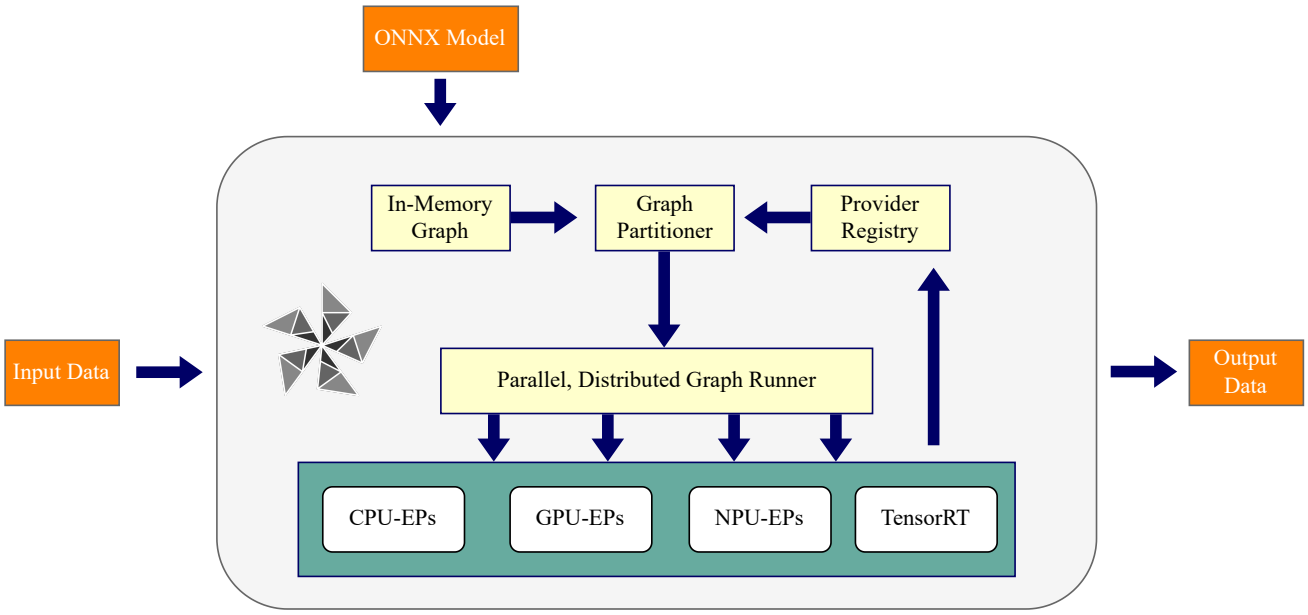
Về quy trình hoạt động, ONNX Runtime nhận đầu vào là các tệp .onnx đã được tạo sẵn. ONNX Runtime sau đó tạo ra phiên làm việc (Inference Session) và thực hiện tuần tự các bước xử lí kĩ thuật sau:

- Tệp .onnx ban đầu chỉ là một chuỗi dữ liệu nhị phân được nén lại để lưu trữ trên ổ cứng. Máy tính không thể tính toán trực tiếp trên chuỗi dữ liệu này. Do đó, bước đầu tiên ORT thực hiện là đọc tệp tin và xây dựng lại toàn bộ cấu trúc mạng lên bộ nhớ thực thi (RAM/VRAM) dưới dạng một cấu trúc dữ liệu liên kết. Trong cấu trúc này, các phép toán và luồng dữ liệu được xác định tại các địa chỉ bộ nhớ cụ thể và kết nối trực tiếp với nhau. Việc chuyển đổi này giúp trình thực thi có thể truy cập, phân tích và sửa đổi bất kì thành phần nào của mô hình một cách tức thì.
- Sau khi đồ thị đã được dựng lên bộ nhớ, ORT áp dụng các kĩ thuật tối ưu để tinh chỉnh lại cấu trúc mạng nhằm giảm thiểu khối lượng tính toán, tối ưu tốc độ thực thi mô hình.

- Tiếp theo, hệ thống thực hiện bước phân hoạch đồ thị (Graph Partitioning), tức là chia nhỏ đồ thị lớn thành các phần khác nhau và phân phối cho các tài nguyên phần cứng xử lí và thực thi.

Để tối đa hóa hiệu suất, ONNX Runtime áp dụng ba cơ chế chính trong quá trình thực thi:

- Ánh xạ phần cứng (Hardware Mapping): Hệ thống tự động chuyển đổi các toán tử trừu tượng của mô hình thành các lệnh điều khiển cụ thể tương ứng với từng loại vi xử lí. Ví dụ, nếu phát hiện phần cứng là CPU, ORT sẽ sử dụng các tập lệnh tính toán tối ưu cho CPU. Nếu có GPU, nó sẽ gọi các nhân tính toán (kernels) chuyên dụng của GPU để xử lý các phép nhân ma trận song song.
- Song song hóa (Parallelism): ORT quản lý các luồng dữ liệu (threads) để tận dụng kiến trúc đa nhân của máy tính. Hệ thống sẽ tự động phát hiện các nhánh tính toán không phụ thuộc lẫn nhau trong đồ thị để cho phép chúng chạy cùng một lúc, thay vì phải chạy lần lượt từng cái một, giúp giảm đáng kể thời gian chờ đợi.
- Thực thi hỗn hợp (Hybrid Execution): Trong trường hợp các thiết bị phần cứng tăng tốc như NPU hay GPU không hỗ trợ toàn bộ các phép toán trong mô hình, ORT sẽ tự động chia tách quy trình xử lí. Các phép toán được hỗ trợ sẽ chạy trên bộ tăng tốc để đạt tốc độ cao, trong khi các phép toán còn lại sẽ được chuyển về chạy trên CPU. Cơ chế này đảm bảo mô hình luôn hoạt động được trên mọi cấu hình phần cứng.

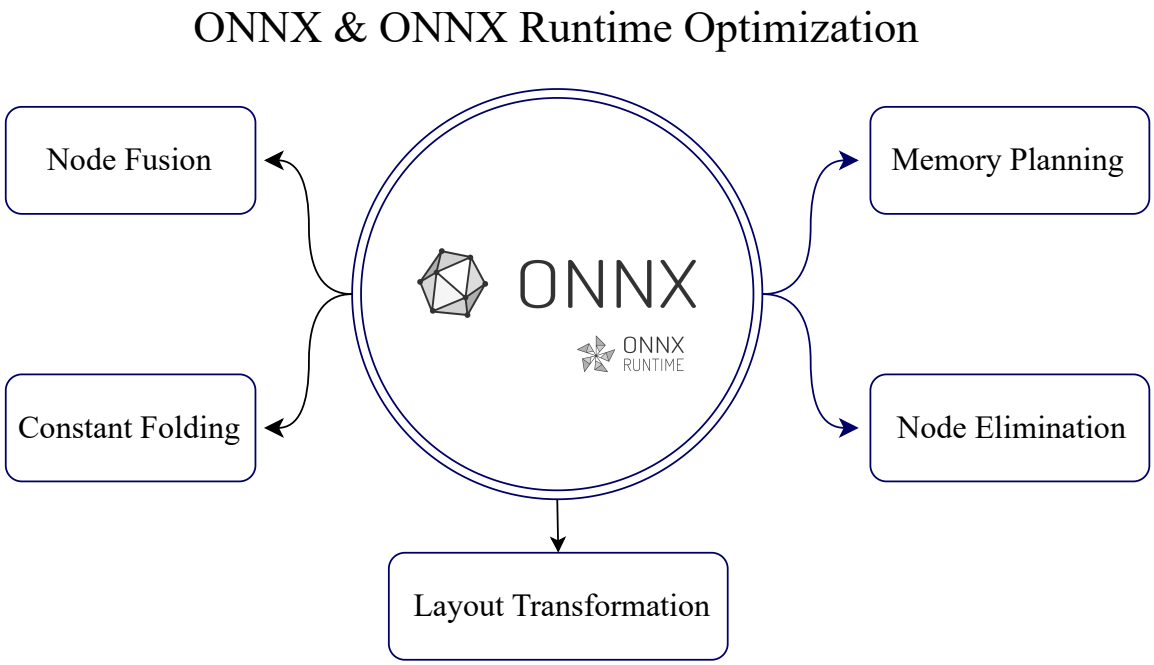


Hình 8: Kiến trúc ONNX Runtime.

III.2.2. Các kĩ thuật tối ưu hóa trong ONNX và ONNX Runtime

Trong quy trình tối ưu hóa, ONNX thường tập trung vào tối ưu hóa kiến trúc đồ thị, trong khi ONNX Runtime tập trung tối ưu tận dụng tài nguyên phần cứng và tăng tốc độ thực thi mô hình.

Node Fusion: Trong phần cứng máy tính, việc di chuyển dữ liệu giữa bộ nhớ và vi xử lí thường tốn thời gian hơn nhiều so với việc thực hiện các phép tính. Trong các mô hình học sâu, thường xuất hiện các chuỗi phép toán diễn ra liên tiếp, ví dụ như một lớp Convolution thường được theo sau bởi lớp Normalization và Activation Function, buộc hệ thống phải thực hiện đọc dữ liệu, tính toán, rồi ghi kết quả trung gian trở lại bộ nhớ ba lần. Quá trình này tạo ra một nút thắt băng thông, ảnh hưởng trực tiếp đến thời gian thực thi mô hình. Thay vì thực thi ba thao tác riêng biệt gây tốn kém chi phí, trình tối ưu hóa của ONNX sẽ kết hợp các toán tử này lại thành một toán tử duy nhất. Dữ liệu sẽ được giữ lại và tính toán tiếp ngay trong bộ nhớ đệm (cache) rồi mới ghi xuống bộ nhớ một lần duy nhất. Cách tiếp cận này giảm thiểu tối đa độ trễ do truy xuất bộ nhớ và tận dụng triệt để sức mạnh của các bộ tính toán phần cứng.



Hình 9: Các kĩ thuật tối ưu hóa trong ONNX và ONNX Runtime.

Constant Folding: Bên cạnh việc tối ưu luồng dữ liệu, tính chất tĩnh của đồ thị còn cho phép áp dụng kĩ thuật Constant Folding để giảm tải khối lượng tính toán thực tế. Trong các lớp chuẩn hóa như Batch Normalization, thường xuất hiện các phép toán phụ trợ dựa trên những tham số cố định đã được huấn luyện. Ví dụ, để chuẩn hóa dữ liệu, hệ thống thường phải tính toán căn bậc hai của tổng phương sai và một hằng số nhỏ ϵ (công thức $\sqrt{\sigma^2 + \epsilon}$). Nếu không có tối ưu hóa, phép tính căn bậc hai này sẽ bị lặp lại liên tục mỗi khi có dữ liệu mới đi qua. Với Constant Folding, ONNX Runtime sẽ nhận diện rằng σ và ϵ là các giá trị bất biến, từ đó thực hiện phép tính này ngay trong quá trình biên dịch mô hình, thay thế toàn bộ cụm công thức phức tạp trên bằng một giá trị kết quả duy nhất được nhúng trực tiếp vào tệp tin.

Redundant Node Elimination: ONNX còn thực hiện việc làm sạch đồ thị thông qua cơ chế loại bỏ các node dư thừa. Trong quá trình chuyển đổi từ môi trường huấn luyện sang môi trường thực thi, đồ thị thường mang theo các phép toán không còn giá trị sử dụng cho việc đưa ra dự đoán. Điển hình nhất là các lớp Dropout, một kĩ thuật quan trọng để tránh hiện tượng overfitting khi huấn luyện nhưng hoàn toàn vô nghĩa khi chạy thực tế. Trình tối ưu hóa sẽ nhận diện và loại

bỏ hoàn toàn các node này khỏi đồ thị, nối trực tiếp đầu ra của lớp trước vào đầu vào của lớp sau. Việc này không chỉ giảm kích thước tệp tin mô hình mà còn loại bỏ các thao tác sao chép bộ nhớ không cần thiết, giúp luồng dữ liệu di chuyển liền mạch hơn.

Memory Planning: Nhờ nắm rõ toàn bộ sơ đồ tính toán trước khi vận hành, ONNX Runtime có khả năng thực hiện quy hoạch bộ nhớ dựa trên vòng đời của dữ liệu. Ví dụ trong quá trình xử lí ảnh, khi lớp tích chập thứ nhất (Layer 1) hoàn tất việc chuyển dữ liệu sang lớp thứ hai (Layer 2), các feature maps của Layer 1 sẽ không còn được sử dụng nữa. Trong cơ chế thông thường, dữ liệu này vẫn chiếm dụng RAM cho đến khi một bộ thu gom tiến hành dọn dẹp. Tuy nhiên, với cơ chế Memory Planning, ONNX Runtime sẽ chỉ định lớp thứ ba (Layer 3) ghi đè kết quả trực tiếp lên vùng nhớ vật lí mà Layer 1 vừa giải phóng. kĩ thuật này giúp tái sử dụng vùng nhớ đệm, giảm thiểu đáng kể dung lượng bộ nhớ, cho phép vận hành các mô hình sâu và phức tạp ngay cả trên các thiết bị biên có tài nguyên hạn chế.

Layout Transformation Là một kĩ thuật nâng cao khác được áp dụng để tác động trực tiếp đến tốc độ xử lí trên phần cứng. Các bộ xử lý khác nhau thường ưu tiên các cách sắp xếp bộ nhớ khác nhau để đạt hiệu suất tối đa. Ví dụ, CPU thường xử lý tốt nhất với định dạng $NCHW$ (Batch - Channel - Height - Width), trong khi một số kiến trúc GPU hoặc các bộ gia tốc phần cứng chuyên biệt lại tối ưu hơn với định dạng $NHWC$ (Batch - Height - Width - Channel) do đặc thù của các lõi tính toán tensor. Trình tối ưu hóa của ONNX Runtime có khả năng tự động phân tích và chèn các phép toán chuyển vị (Transpose) hoặc thay đổi cấu trúc dữ liệu bên trong của đồ thị để khớp với định dạng tối ưu của phần cứng mục tiêu. Sự tương này giúp tối đa hóa băng thông và giảm thiểu nút thắt cổ chai khi dữ liệu được nạp từ RAM vào các vi xử lí.

Các cách tiếp cận này mang lại tính linh hoạt tối đa: tệp mô hình .onnx giữ vai trò trung lập và độc lập với phần cứng. Khi mô hình được nạp vào một thiết bị cụ thể, trình thực thi sẽ tự động phân tích cấu hình phần cứng để áp dụng các chiến lược tối ưu hóa đồ thị phù hợp nhất cho thiết bị đó. Điều này đảm bảo rằng cùng một tệp mô hình có thể hoạt động với hiệu suất cao nhất trên đa dạng các nền tảng khác nhau mà không cần phải thực hiện chuyển đổi lại từ đầu.

III.2.3. Providers

Trong kiến trúc phân tầng của ONNX Runtime, Execution Providers (EP) đóng vai trò là lớp giao tiếp với phần cứng. Thành phần này chịu trách nhiệm đóng gói các thư viện tính toán cấp thấp thành một định dạng chuẩn hóa, cho phép ONNX Runtime giao tiếp và điều khiển tài nguyên phần cứng mà không cần can thiệp vào chi tiết kĩ thuật bên dưới. Cơ chế trừu tượng hóa này giúp tách biệt hoàn toàn tầng logic thuật toán khỏi tầng thực thi vật lí, đảm bảo tính di động của ứng dụng trên đa dạng các nền tảng phần cứng khác nhau.

Cơ chế vận hành của Execution Providers dựa trên nguyên tắc đăng ký và phân phối:

- Khi một phiên làm việc được thiết lập, hệ thống yêu cầu khai báo danh sách các Execution Providers (EPs) mục tiêu theo thứ tự ưu tiên. Ví dụ, trong môi trường có GPU, thứ tự ưu tiên thường là *TensorRT Execution Provider* $\rightarrow$ *CUDA Execution Provider* $\rightarrow$ *CPU Execution Provider*.
- ONNX Runtime sau đó thực hiện duyệt toàn bộ đồ thị tính toán. Tại mỗi node, hệ thống sẽ kiểm tra khả năng hỗ trợ của các EP theo thứ tự đã đăng ký. Nếu một EP xác nhận hỗ

trợ toán tử tại node đó, quyền thực thi node sẽ được gán cho EP tương ứng. Quá trình này tạo ra sự phân hoạch đồ thị, trong đó các phần khác nhau của mô hình được xử lí bởi các đơn vị phần cứng khác nhau.

- Để đảm bảo tính toàn vẹn của quá trình thực thi, ONNX Runtime duy trì cơ chế dự phòng tự động. Trong trường hợp EP có độ ưu tiên cao không hỗ trợ một toán tử cụ thể, hệ thống sẽ tự động chuyển quyền xử lí toán tử đó cho EP có độ ưu tiên thấp hơn, mặc định thường là CPU Execution Provider. Điều này đảm bảo mô hình luôn vận hành ổn định mà không bị gián đoạn do các vấn đề tương thích cục bộ.

Các Execution Providers phổ biến được tích hợp trong ONNX Runtime bao gồm:

- **CPU Execution Provider (Mặc định):** Sử dụng các thư viện tối ưu hóa toán học như MLAS và oneDNN để khai thác các tập lệnh vector (như AVX, AVX-512) trên vi xử lí trung tâm. Đây là thành phần đảm bảo độ ổn định nền tảng cho mọi mô hình.
- **CUDA Execution Provider:** Cung cấp khả năng tăng tốc phần cứng trên các dòng GPU của NVIDIA. EP này thực hiện ánh xạ các toán tử ONNX sang các kernels tính toán song song của thư viện cuDNN và cuBLAS.
- **TensorRT Execution Provider:** Là giải pháp tối ưu hóa chuyên sâu dành cho NVIDIA GPU. Khác với CUDA EP chỉ thực thi các hạt nhân riêng lẻ, TensorRT EP sử dụng trình biên dịch TensorRT để đạt hiệu suất suy luận tối đa.
- **OpenVINO Execution Provider:** Được thiết kế tối ưu cho hệ sinh thái phần cứng của Intel (bao gồm CPU, iGPU và VPU), giúp khai thác triệt để kiến trúc vi xử lí của hãng này.

III.3. Thực hành ONNX

Trong phần này, chúng ta sẽ thực hành chuyển đổi mô hình ResNet50 từ Pytorch thành định dạng ONNX, sử dụng trình tích hợp sẵn. Sau đó thực hiện Inference trên ONNX Runtime.

III.3.1. Chuyển đổi mô hình sang ONNX.

Đầu tiên cài đặt các thư viện cần thiết bao gồm onnx, onnxruntime-gpu và onnxscript.

```
!pip install -q torch torchvision onnx onnxruntime-gpu onnxscript tqdm
```

Tải mô hình ResNet50 và chuyển sang chế độ eval.

```
import torch
import torchvision.models as models

torch_model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V1)
torch_model.eval()
```

Khai báo đường dẫn lưu mô hình, kích thước dữ liệu và Opset.

```
ONNX_PATH = "./resnet50.onnx"
IMG_SIZE = 224
OPSET = 18
```

Bây giờ, tiến hành chuyển đổi mô hình sang định dạng ONNX. Đầu tiên, ta khai báo một dummy_input truyền vào ONNX để thực hiện tracing.

```
dummy_input = torch.randn(1, 3, IMG_SIZE, IMG_SIZE)
```

Để chuyển đổi sang ONNX, Pytorch hỗ trợ sẵn công cụ tích hợp trong thư viện torch. Ta sẽ dùng phương thức export() trong torch.onnx.

```
torch.onnx.export(
    torch_model,
    dummy_input,
    ONNX_PATH,
    export_params=True, #Lưu kèm trọng số vào file.
    opset_version=OPSET,
    do_constant_folding=True, #Tối ưu hóa hằng số.
    input_names=["input"], # Đặt tên cho node đầu vào
    output_names=["output"], # Đặt tên cho node đầu ra
    #dynamic_axes = ... # Tùy chọn Dynamic Shape
)
```

Kết quả sau khi chạy xong là một file restnet50.onnx được chuyển đổi từ mô hình Pytorch. Ta sẽ sử dụng file này để inference trên các trình hiệu năng cao như ONNX Runtime hay TensorRT.

Vấn đề khi thực hiện Tracing: Đối với các mô hình chứa các điều kiện rẽ nhánh, sử dụng dummy_input có thể khiến quá trình truy vết luôn đi vào một nhánh mà bỏ qua các nhánh điều kiện còn lại. Khiến logic của mô hình bị thiếu trong đồ thị truy vết. Lúc này, giải pháp thường dùng là:

- Viết lại code mô hình ONNX hiểu được cả hai nhánh.
- Dùng công cụ torch.jit.script thay vì Tracing, đây là cách thường dùng hơn.

Do điểm hạn chế này nên ta phải lựa chọn kĩ mô hình và phương pháp phù hợp khi sử dụng ONNX.

III.3.2. Sử dụng ONNX Runtime

Import các thư viện cần thiết.

```python
import time
from tqdm import tqdm
import numpy as np
import onnxruntime as ort

SAMPLES = 1000
IMG_SIZE = 224
```

Để dùng ONNX Runtime, ta gọi đối tượng ort để tạo một section làm việc thông qua phương thức ort.InferenceSession().

```python
session_options = ort.SessionOptions()
session_options.graph_optimization_level = ort.GraphOptimizationLevel.ORT_ENABLE_ALL

session = ort.InferenceSession(
    "resnet50.onnx", #Tệp ONNX.
    providers=["CPUExecutionProvider", "CUDAExecutionProvider"],
    sess_options = session_options #Optimization option
)
```

Trong đoạn code trên, sess_options dùng để chọn chiến lược tối ưu hóa. Bao gồm bốn cấp độ tùy chọn:

- ORT_DISABLE_ALL (Level 0): Tắt toàn bộ các kỹ thuật tối ưu hóa. Mô hình sẽ chạy chính xác như đồ thị gốc được xuất ra từ PyTorch. Dùng để debug hoặc đo baseline.
- ORT_ENABLE_BASIC (Level 1): Chỉ thực hiện các tối ưu hóa cơ bản và đảm bảo không làm thay đổi ngữ nghĩa của đồ thị: Constant Folding, Redundant Node Elimination, Node Fusion.
- ORT_ENABLE_EXTENDED (Level 2): Bao gồm toàn bộ Level 1 cộng thêm các kĩ thuật Node Fusion phức tạp hơn. Đây là mức cân bằng tốt nhất thường được dùng.
- ORT_ENABLE_ALL (Level 99): Bao gồm Level 1 + Level 2 + Layout Optimization.

Tham số providers chỉ định các EP phần cứng cụ thể. Trong code trên khai báo [“CPUExecutionProvider”, “CUDAExecutionProvider”] nghĩa là mô hình sẽ ưu tiên chạy trên GPU.

Sau khi khai báo thành công phiên làm việc. Tiến hành inference thông qua phương thức session.run(). Ở đây với mục đích đo tốc độ inference, ta tạo các dữ liệu ngẫu nhiên để đưa vào mô hình.

```python
onnx_total_time = 0.0

for _ in tqdm(range(SAMPLES)):
    x = np.random.randn(1, 3, IMG_SIZE, IMG_SIZE).astype(np.float32)
```

```
    start_time = time.time()
    y = session.run(None, {"input": x})
    end_time = time.time()
    onnx_total_time += (end_time - start_time)

print(f"ONNX Total Inference Time for {SAMPLES} samples: {onnx_total_time:.4f} seconds"
                                                        )
```

Trong sesstion.run() ta truyền vào hai tham số như sau:

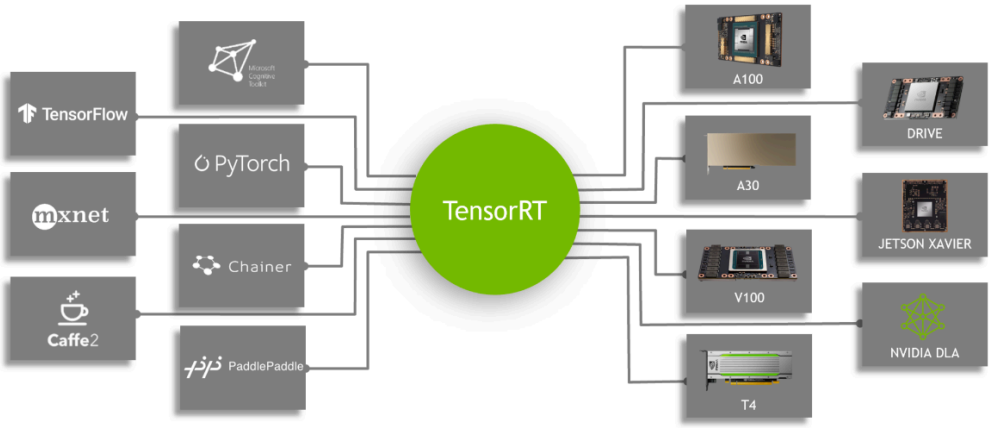
- output_names = None: Một mô hình có thể tra ra nhiều output. Tham số này yêu cầu liệt kê danh sách tên các đầu ra muốn nhận về, ví dụ: [“Bounding Boxes”, “Confidence Scores”, “Class Probabilities”] cho mô hình YOLO.
- {“input”: x}: Dữ liệu đầu vào.

Sau khi chạy xong, kết quả sẽ in ra thời gian Inference toàn bộ sample theo đơn vị giây.

IV. Section 3: TensorRT

IV.1. Tổng quan về TensorRT

TensorRT là một trình biên dịch hiệu năng cao dành cho các mô hình học sâu được phát triển bởi NVIDIA nhằm giải quyết bài toán tối ưu trong các ứng dụng AI thực tế. TensorRT đóng vai trò chuyển đổi các mô hình học sâu thành các tệp tin gọi là engine có khả năng thực thi tối ưu. Điểm nổi bật của TensorRT là khả năng khai thác triệt để kiến trúc phần cứng của NVIDIA, không chỉ giới hạn ở các nhân GPU (CUDA Core, Tensor Core) mà còn mở rộng sang các bộ gia tốc chuyên biệt như DLA (Deep Learning Accelerator) có trên các phần cứng như NVIDIA Jetson hay NVIDIA DRIVE.



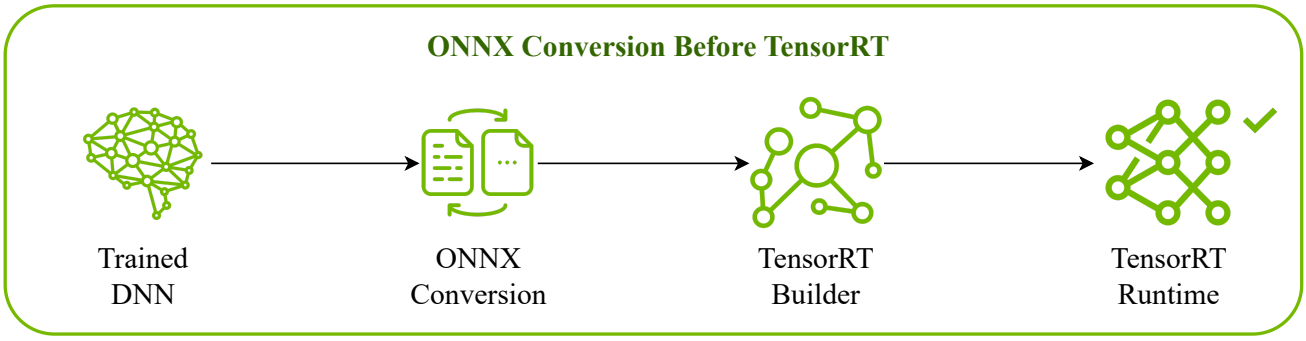
Hình 10: Khả năng triển khai của TensorRT trên các phần cứng NVIDIA.

Tuy nhiên, không chỉ dừng lại ở một công cụ biên dịch đơn thuần, NVIDIA đã mở rộng TensorRT thành một hệ sinh thái đa dạng nhằm giải quyết các bài toán triển khai AI chuyên biệt, bao gồm:

- **TensorRT Core SDK:** Là thành phần nền tảng, bao gồm Builder chịu trách nhiệm tối ưu hóa phần cứng và biên dịch mô hình và Runtime để tạo môi trường thực thi engine gọn nhẹ trên thiết bị đích.
- **TensorRT-LLM:** Thư viện chuyên biệt dành cho các mô hình ngôn ngữ lớn, tích hợp các kĩ thuật tiên tiến để thực thi các mô hình có trọng số khổng lồ và tối ưu hóa tính toán song song trên đa GPU.
- **TensorRT Model Optimizer:** Công cụ nén mô hình chuyên sâu hoạt động trước bước biên dịch. Sử dụng các kĩ thuật như lượng tử hóa (Quantization) và cắt tỉa (Pruning) để giảm kích thước và độ phức tạp tính toán của mô hình gốc mà vẫn duy trì độ chính xác.
- **TensorRT for Windows (RTX/GTX):** Phiên bản tối ưu hóa cho hệ điều hành Windows và các dòng card GeForce RTX, giúp tích hợp AI vào các ứng dụng PC với độ trễ thấp nhất.
- **TensorRT Cloud:** Triển khai mô hình trên Cloud.

Tùy thuộc vào yêu cầu cụ thể về hiệu năng và độ phức tạp của hệ thống, TensorRT cung cấp ba luồng làm việc chính để tích hợp vào ứng dụng thực tế:

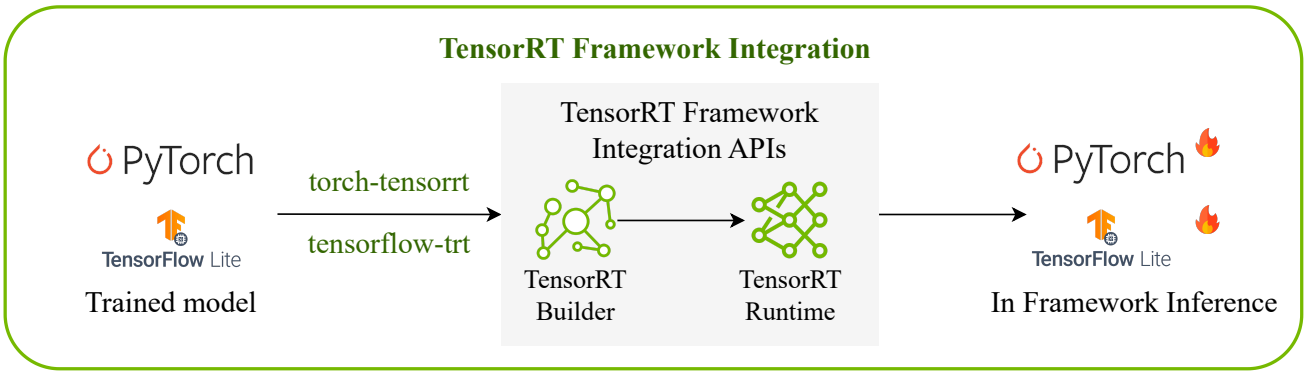
1. Naive TensorRT thông qua ONNX: Đây là phương thức triển khai phổ biến và tối ưu nhất về hiệu năng. Quy trình bao gồm việc xuất mô hình từ framework gốc sang định dạng trung gian ONNX, sau đó sử dụng công cụ trtexec hoặc API của TensorRT (C++/Python) để biên dịch thành tệp tin TensorRT Engine (.engine) và thực thi.



Hình 11: Naive TensorRT thông qua ONNX.

- *Ưu điểm:* Hiệu suất cao, latency thấp, memory footprint ít do không cần tải bộ thư viện PyTorch/TensorFlow nặng nề khi chạy thực tế.
- *Nhược điểm:* Quy trình phức tạp hơn do phải quản lý việc tiền xử lí và hậu xử lí dữ liệu một cách thủ công.

2. Tích hợp trực tiếp vào Framework (In-Framework Integration): NVIDIA cung cấp các thư viện tích hợp sẵn như torch-tensorrt dành cho PyTorch hay tensorflow-trt cho TensorFlow. Khi sử dụng phương pháp này, TensorRT hoạt động như một thư viện con bên trong framework. Nó sẽ tự động quét đồ thị mô hình để chọn ra thành phần có thể tối ưu bằng TensorRT, các thành phần còn lại sẽ được đẩy về framework gốc để xử lý.

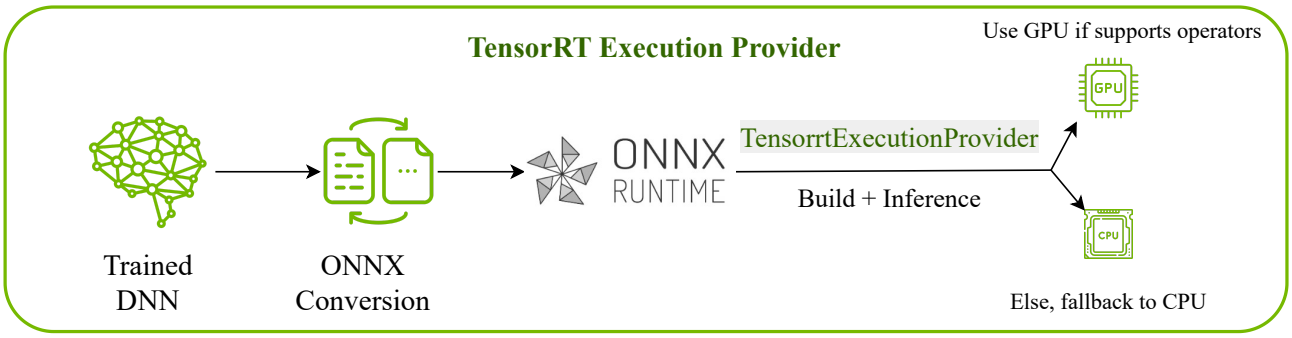


Hình 12: Sử dụng TensorRT thông qua thư viện tích hợp sẵn trong framework.

- *Ưu điểm:* Cực kỳ dễ sử dụng, chỉ cần thêm vài dòng code, đảm bảo mô hình luôn chạy được nhờ cơ chế dự phòng.
- *Nhược điểm:* Hiệu năng không cao bằng Naive TensorRT do vẫn phải gánh chi phí vận hành của framework gốc.

3. ONNX Runtime với TensorRT Execution Provider: Đây là giải pháp hydrid, tận dụng tính linh hoạt của ONNX Runtime và sức mạnh của TensorRT. Người dùng chạy mô hình bằng API chuẩn của ORT nhưng cấu hình provider *TensorrtExecutionProvider* làm backend ưu tiên.

- *Ưu điểm:* Cân bằng tốt giữa hiệu năng và tính tiện dụng. Nếu TensorRT gặp lỗi ở một toán tử lạ, ORT sẽ tự động chuyển toán tử đó sang CUDA hoặc CPU mà không làm sập chương trình.
- *Ứng dụng:* Thường được sử dụng trong các ứng dụng cần hỗ trợ đa dạng phần cứng mà không muốn viết lại code cho từng loại chip.

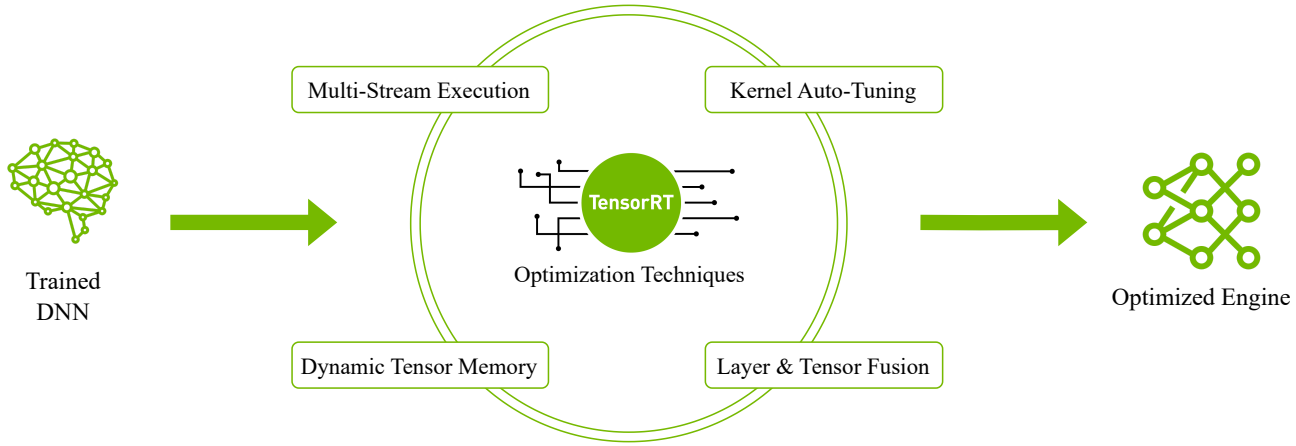


Hình 13: Sử dụng TensorRT kết hợp với ONNX-Runtime với khai báo Tensorrt Execution Provider.

Lưu ý, TensorRT Engine không có tính di động. Một engine được build trên GPU A sẽ không chạy được hoặc chạy sai trên GPU B nếu khác kiến trúc. Nó cũng phụ thuộc chặt chẽ vào phiên bản TensorRT và Driver. Do tính chất ràng buộc trên, có 2 chiến lược triển khai:

- **Triển khai trên máy người dùng (Client-side):** Phân phối tệp ONNX. Ứng dụng sẽ tự động build ra Engine ngay trên máy người dùng trong lần chạy đầu tiên để đảm bảo tương thích tuyệt đối.
- **Triển khai trên Server (Server-side):** Nếu quản lý một cụm server có phần cứng giống hệt nhau, ta có thể build engine một lần rồi copy sang các máy còn lại để tiết kiệm thời gian.

IV.2. Các kĩ thuật tối ưu hóa trong TensorRT

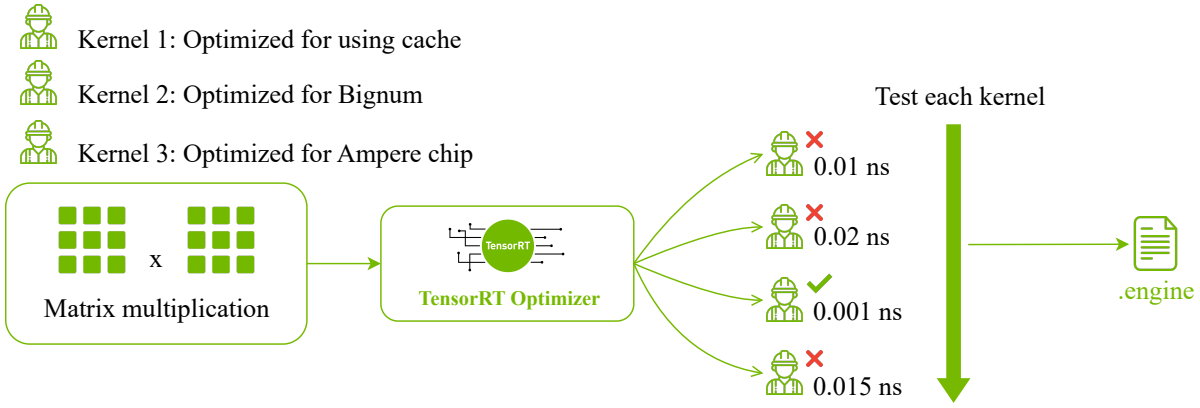


Hình 14: Các kĩ thuật tối ưu hóa trong TensorRT.

IV.2.1. Kernel Auto -Tuning

Trong tính toán hiệu năng cao, một toán tử toán học có thể được hiện thực hóa bằng nhiều thuật toán lập trình khác nhau. Mỗi phiên bản cài đặt cụ thể cho một thuật toán như vậy được gọi là một kernel. Ví dụ, cùng là phép nhân ma trận, nhưng có thể tồn tại hàng chục kernel khác nhau, có kernel tối ưu cho ma trận kích thước nhỏ, có kernel lại tận dụng tốt bộ nhớ đệm cho ma trận lớn, hoặc có kernel chuyên biệt để chạy trên các lõi Tensor Core.

Tuy nhiên, tại thời điểm thực thi, mỗi phép toán chỉ có thể được xử lý bởi một kernel duy nhất. Lúc này, vấn đề đặt ra là làm thế nào để chọn đúng kernel tốt nhất trong hàng loạt các kernel khả thi. Đây chính là lúc cơ chế Kernel Auto-Tuning của TensorRT phát huy tác dụng.



Hình 15: Kĩ thuật Kernel Auto-Tuning chọn ra kernel tối ưu cho toán tử.

Thay vì sử dụng một kernel mặc định cho mọi tình huống như các framework thông thường, TensorRT thực hiện quy trình dò tìm dựa trên thực nghiệm:

- Hệ thống quét qua thư viện kernel từ hệ thống NVIDIA.

- Với mỗi lớp mạng, TensorRT nạp dữ liệu giả lập và chạy thử nghiệm trực tiếp tất cả các kernel khả thi trên thiết bị đích.
- TensorRT đo lường thời gian thực thi chính xác và chọn ra kernel có độ trễ thấp nhất để ghi vào tệp .engine cuối cùng.

Đây là lí do khi build engine với TensorRT, ta cần chờ một khoản thời gian để hệ thống thử nghiệm các kernel.

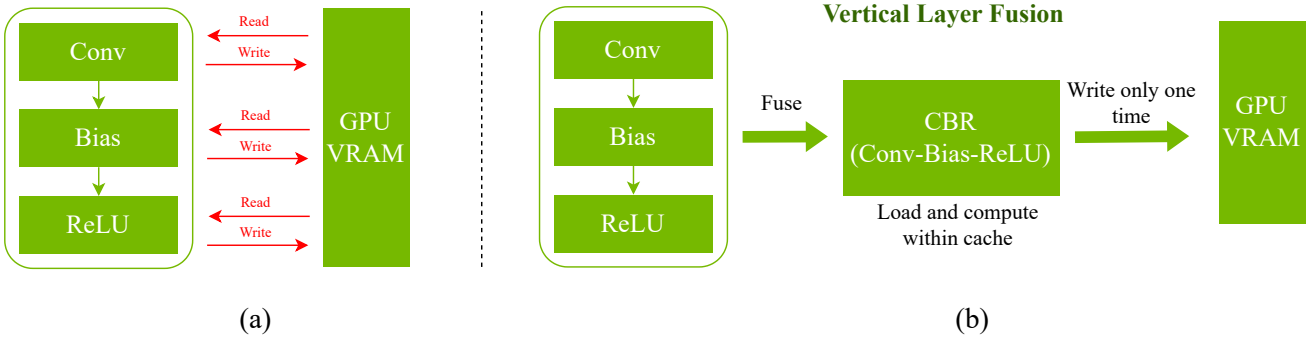
IV.2.2. Layer và Tensor Fusion

Trong kiến trúc máy tính tồn tại một nút thắt cổ chai giữa tốc độ tính toán siêu nhanh của các vi xử lí và tốc độ đọc ghi dữ liệu chậm chạp của hệ thống. Hiện tượng này thường được gọi là Memory Wall. Khi một mô hình học sâu được tính toán, các tham số và dữ liệu liên tục được đọc từ VRAM sang bộ nhớ tính toán tại vi xử lí. Hậu quả là vi xử lí dù mạnh mẽ đến đâu cũng thường xuyên rơi vào trạng thái đói dữ liệu (data starvation), tức là phải nằm chờ dữ liệu được nạp vào thay vì thực hiện phép tính. Đây chính là điểm nghẽn hiệu năng lớn nhất, gây lãng phí tài nguyên và làm tăng độ trễ của hệ thống.

Để giải quyết bài toán này, TensorRT áp dụng kỹ thuật Layer và Tensor Fusion như một giải pháp nhằm giảm thiểu tối đa tần suất truy cập bộ nhớ toàn cục.

Layer Fusion, hay hợp nhất lớp, là kĩ thuật cấu trúc lại đồ thị bằng cách gộp nhiều toán tử rời rạc thành một kernel tính toán duy nhất, giảm thiểu độ trễ khi khởi động kernel và giảm số lần đọc ghi bộ nhớ. Layer Fusion được chia thành hai dạng chính:

1. **Vertical Fusion:** Gộp các lớp mạng được được thực thi tuần tự lại với nhau. Ví dụ, trong tính toán thông thường của các mạng Convolutional Neural Network (CNN), chuỗi ba lớp: Convolution → Batch Normalization → ReLU được thực hiện tính toán lần lượt:
   (a) Đọc input, tính convolution, ghi kết quả trung gian T_1 xuống VRAM.
   (b) Đọc T_1, tính batch norm, ghi kết quả trung gian T_2 xuống VRAM.
   (c) Đọc T_2, áp dụng ReLU, ghi kết quả cuối cùng xuống VRAM.

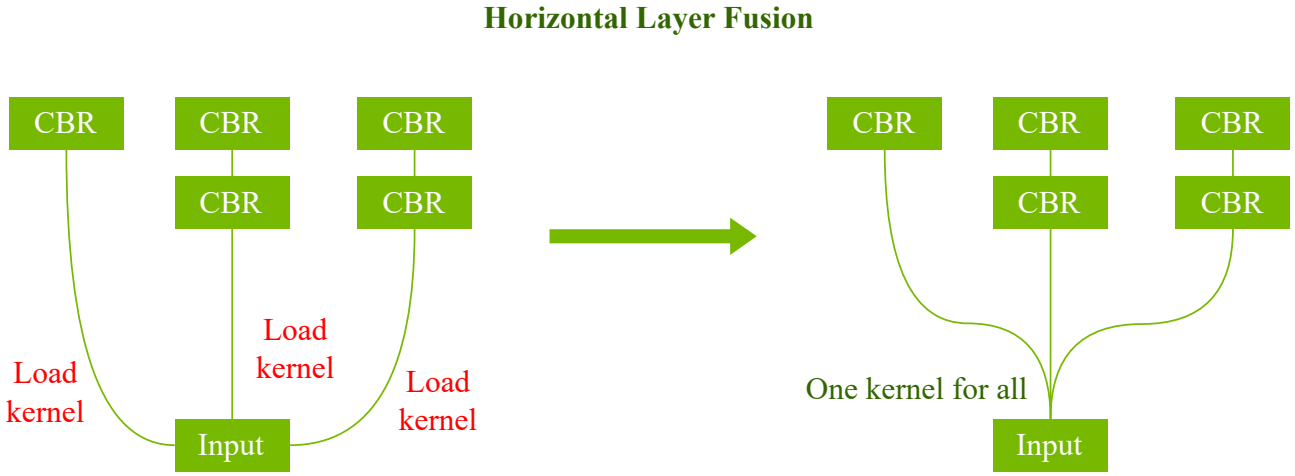


Hình 16: (a) Vấn đề đọc ghi dữ liệu, (b) Vertical Layer Fusion kết hợp các layer, thực hiện tính toán toàn bộ trong cache rồi mới ghi vào bộ nhớ.

Với Vertical Fusion, TensorRT tạo ra một kernel tích hợp duy nhất (thường gọi là CBR kernel). Dữ liệu sau khi tính tích chập sẽ được giữ lại ngay trong các thanh ghi (registers) hoặc bộ nhớ đệm tốc độ cao (L1 Cache) để thực hiện tiếp phép cộng bias, chuẩn hóa và kích hoạt trước khi ghi kết quả cuối cùng ra bộ nhớ. Quy trình này loại bỏ hoàn toàn các bước đọc, ghi trung gian thừa thãi, giảm thời gian chờ dữ liệu của vi xử lí.

2. **Horizontal Fusion:** Kết hợp các lớp không phụ thuộc vào nhau có cùng một đầu vào. Trong CNN, tại một điểm rẽ nhánh, tensor đầu vào có thể được đưa đồng thời qua các lớp Convolution khác nhau để trích xuất các đặc trưng riêng biệt. Thông thường, GPU sẽ phải xử lí lần lượt từng nhánh một. Vì mỗi nhánh chỉ thực hiện khối lượng tính toán nhỏ, nên chỉ một phần nhỏ các lõi xử lí được dùng để tính toán trong khi phần lớn các lõi còn lại lại trong trạng thái chờ. Việc này gây ra sự lãng phí tài nguyên khổng lồ.

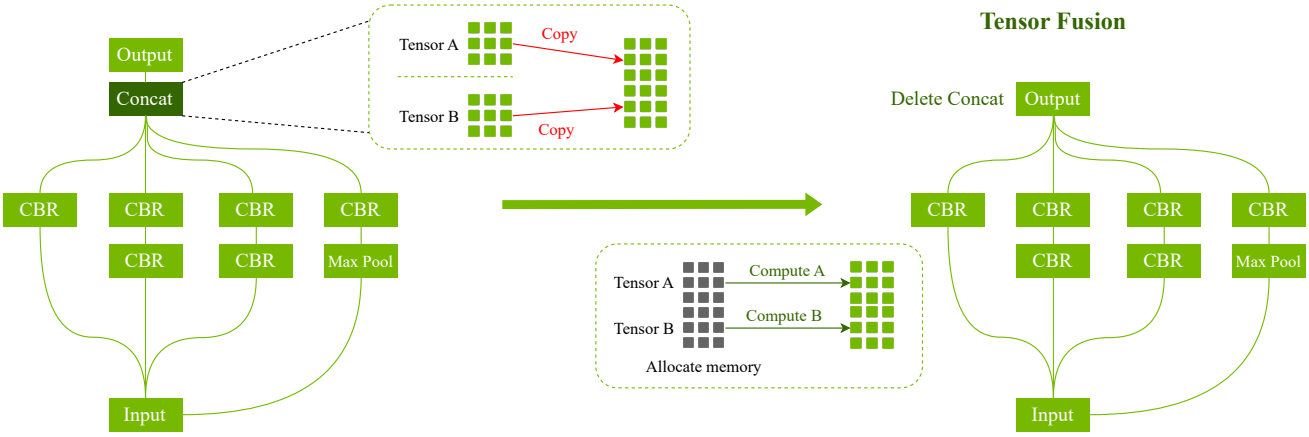
   TensorRT tối ưu bằng cách kết hợp các phép toán của hai nhánh lại thành một kernel lớn duy nhất. Giống như cách chúng ta gom đủ hàng để chở một chuyến xe lớn thay vì chạy nhiều chuyến xe rỗng. Kĩ thuật này giúp huy động số lượng lớn các lõi GPU cùng tham gia tính toán song song, đảm bảo phần cứng vận hành ở hiệu suất cao nhất.



Hình 17: Horizontal Layer Fusion kết hợp các layer giống nhau có cùng input thành một kernel duy nhất.

Tensor Fusion: Bên cạnh Layer Fusion, TensorRT còn tối ưu hóa cách thức lưu trữ và di chuyển dữ liệu thông qua cơ chế Tensor Fusion. Trong các mạng CNN, lớp Concat thường xuyên xuất hiện để ghép nối các feature maps lại với nhau. Về mặt logic, Concat không thực hiện phép tính toán học nào mà chỉ đơn thuần là sao chép dữ liệu từ các vùng nhớ rời rạc vào một vùng nhớ liên tục mới. Thao tác sao chép này gây lãng phí băng thông bộ nhớ.

Lúc này, Tensor Fusion sẽ thực hiện loại bỏ lớp concat này. Thay vì chờ dữ liệu sinh ra rồi mới sao chép để ghép lại, TensorRT cấp phát sẵn một vùng nhớ chung cho kết quả cuối cùng. Các lớp thành phần sẽ được chỉ dẫn để ghi kết quả thẳng vào đúng vị trí tương ứng trong vùng nhớ đã được cấp phát trước đó. Nhờ vậy, dữ liệu tự động được ghép nối ngay khi vừa được tính toán xong, loại bỏ hoàn toàn bước sao chép trung gian tốn kém. Đối với các lớp tương tự như concat, Tensor Fusion cũng sẽ thực hiện loại bỏ để tối ưu thời gian thực thi mô hình.

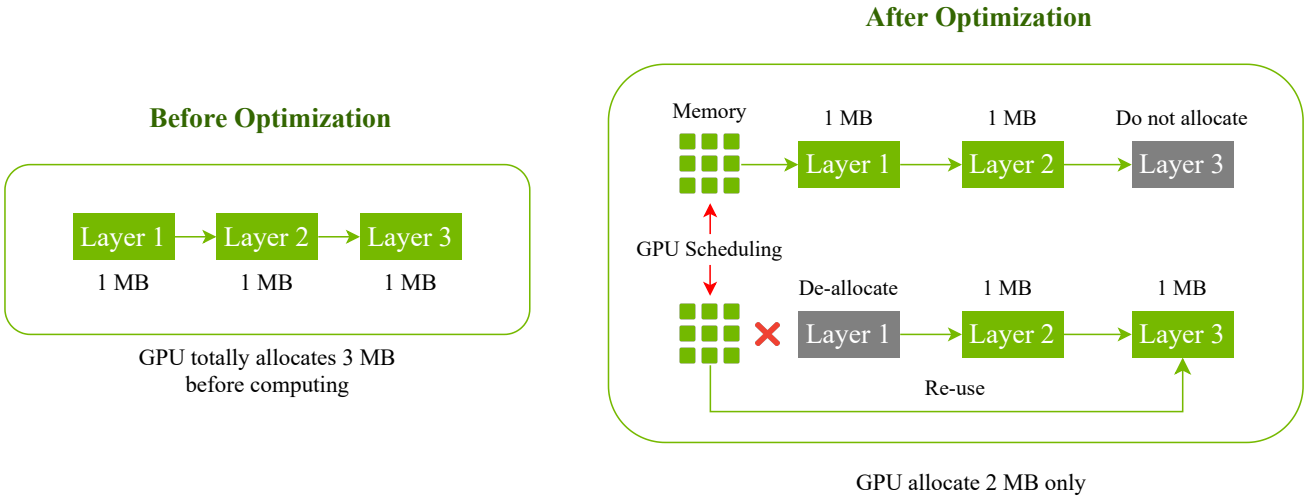


Hình 18: Kĩ thuật Tensor Fusion.

IV.2.3. Dynamic Tensor Memory

Một thách thức lớn khi triển khai các mô hình học sâu là vấn đề tiêu thụ bộ nhớ, hay Memory Footprint. Mỗi lớp mạng của mô hình sẽ yêu cầu một vùng nhớ riêng để lưu trữ kết quả tính toán trung gian. Đối với các mô hình sâu hàng trăm lớp, việc cấp phát liên tục này sẽ nhanh chóng làm tràn bộ nhớ, ngay cả khi kích thước mô hình gốc không quá lớn. Ngoài ra quá trình Inference chỉ diễn ra theo một chiều nên các kết quả trung gian lúc này không cần thiết phải lưu trữ dài hạn.

TensorRT sẽ thực hiện phân tích vòng đời của từng tensor trong đồ thị tính toán, xác định chính xác thời điểm một tensor được sinh ra và thời điểm nó hết giá trị sử dụng. Dựa trên phân tích này, TensorRT sẽ cấp phát các vùng nhớ chung. Khi một tensor ở layer trước được sử dụng xong, nó sẽ được xóa khỏi vùng nhớ hiện tại, vùng nhớ đó được dùng để cấp phát cho layer sau. Hay nói cách khác, hệ thống sẽ cho phép các tensor không tồn tại cùng một thời điểm được phép dùng chung một địa chỉ bộ nhớ vật lí.



Hình 19: TensorRT sử dụng Dynamic Tensor Memory để cấp phát và thu hồi bộ nhớ liên tục, giúp giảm memory footprint.

Kĩ thuật này giúp giảm thiểu đáng kể dung lượng bộ nhớ cần thiết, giúp TensorRT có thể vận hành các mô hình có kích thước lớn hơn nhiều so với dung lượng vật lí của thiết bị, đồng thời tăng hiệu suất do tận dụng tốt hơn bộ nhớ cache do dữ liệu luôn nằm tại một vùng nhớ xác định.

IV.2.4. Multi-Stream Execution

Trong các kiến trúc GPU hiện đại (như NVIDIA Ampere hay Hopper), số lượng các nhân tính toán (Streaming Multiprocessors - SMs) là cực kỳ lớn. Đối với các mô hình học sâu có kích thước nhỏ hoặc trung bình, việc chạy một suy luận đơn lẻ thường không thể sử dụng hết 100% khả năng của GPU. Điều này dẫn đến tình trạng lãng phí tài nguyên, khi một phần lớn các lõi CUDA bị bỏ trống trong khi chờ đợi tác vụ hiện tại hoàn thành.

Để giải quyết vấn đề này, TensorRT hỗ trợ cơ chế Multi-Stream Execution dựa trên khái niệm CUDA Streams.

Mặc định, các lệnh gửi xuống GPU được xếp vào một hàng đợi duy nhất và thực thi tuần tự, tác vụ B phải chờ Tác vụ A hoàn tất mới được bắt đầu. Với Multi-Stream, TensorRT cho phép khởi tạo nhiều hàng đợi lệnh độc lập. Khi ứng dụng gửi các yêu cầu suy luận khác nhau vào các stream này, bộ lập lịch phần cứng (Hardware Scheduler) của GPU sẽ tự động phân phối chúng xuống các SMs còn trống. Nếu GPU đủ lớn, hai hoặc hoặc nhiều request của cùng một mô hình có thể chạy song song thực sự về mặt vật lí. Ví dụ, stream 1 sử dụng 40% GPU để xử lí request A, stream 2 sử dụng 40% GPU để xử lí request B ngay tại cùng một thời điểm.

Multi-Stream Execution tập trung tăng thông lượng hệ thống. Kỹ thuật này đặc biệt hữu ích trong các hệ thống máy chủ, nơi phải xử lí hàng nghìn yêu cầu bất đồng bộ từ nhiều người dùng cùng lúc. Thay vì bắt người dùng thứ hai phải chờ người dùng thứ nhất xong việc, hệ thống có thể phục vụ cả hai cùng lúc trên một GPU duy nhất.

Để hỗ trợ Multi-Stream, TensorRT cung cấp tính năng Optimization Profiles. Mỗi profile cho phép định nghĩa một không gian bộ nhớ động riêng biệt. Lập trình viên có thể tạo ra nhiều Execution Contexts tương ứng với các profile này và gán mỗi context vào một CUDA Stream riêng. Điều này đảm bảo các luồng tính toán không bị xung đột bộ nhớ và có thể vận hành song song một cách an toàn.

IV.3. trtexec

trtexec là một công cụ dòng lệnh (command-line tool) được cung cấp sẵn trong gói cài đặt của TensorRT. Công cụ này cho phép người dùng thực hiện chuyển đổi các tệp mô hình (thường là ONNX) sang định dạng TensorRT Engine (.engine). Ngoài ra trtexec còn hỗ trợ tạo dữ liệu ngẫu nhiên để chạy thử nghiệm đo đạc chính xác độ trễ và thông lượng của mô hình trên phần cứng hiện tại. Đây là công cụ tiêu chuẩn dùng để xác minh xem một mô hình có tương thích với TensorRT hay không trước khi bắt đầu tích hợp vào ứng dụng.

Người dùng điều khiển trtexec trên terminal thông qua các cờ (flag). Dưới đây là các flag phổ biến nhất:

- `--onnx=<path>`: Đường dẫn đến tệp mô hình ONNX đầu vào.

- `-saveEngine=<path>`: Đường dẫn lưu tệp Engine đầu ra sau khi biên dịch thành công.
- `-fp16`: Kích hoạt chế độ Mixed Precision sử dụng FP16, giúp tăng tốc độ xử lí trên các GPU hỗ trợ.
- `-int8`: Kích hoạt chế độ lượng tử hóa INT8, thường yêu cầu thêm dữ liệu hiệu chỉnh.
- `-verbose`: Hiển thị chi tiết log trong quá trình biên dịch, giúp phát hiện lỗi nếu mô hình không tương thích.
- `-minShapes, -optShapes, -maxShapes`: Ba tham số bắt buộc khi làm việc với mô hình có kích thước đầu vào động (Dynamic Shapes). Chúng định nghĩa kích thước tối thiểu, tối ưu và tối đa mà engine cần hỗ trợ.

Sau khi chạy xong, trtexec sẽ xuất ra báo cáo thống kê. Các chỉ số quan trọng cần quan tâm bao gồm latency (average và P99), throughput và GPU Compute Time đo thời gian thực tế dành cho việc tính toán.

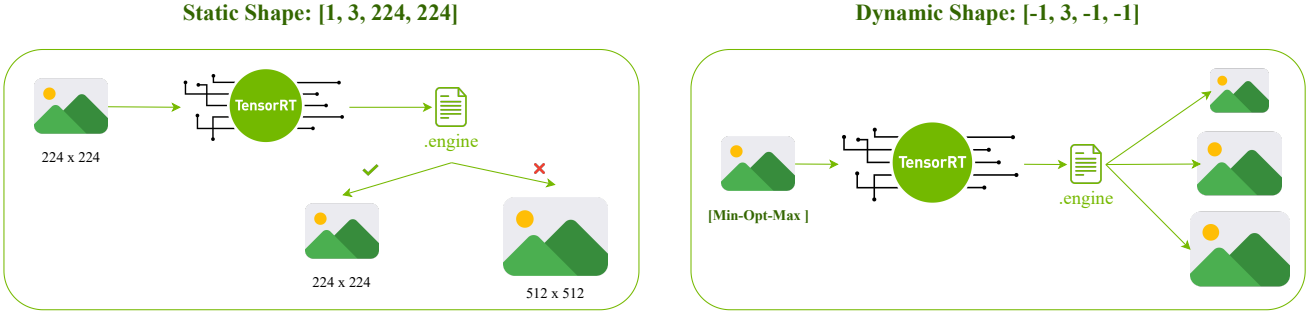
Ví dụ thực hiện chuyển đổi mô hình resnet50.onnx sang engine, kích hoạt chế độ FP16 để tăng tốc và lưu kết quả ra tệp tin.

Ví dụ trtexec

```
!trtexec --onnx=resnet50.onnx --saveEngine=resnet50_fp16.engine --fp16
```

IV.4. Dynamic Shape

Trong các bài toán thực tế, kích thước dữ liệu đầu vào của mô hình thường không cố định. Ví dụ vào những lúc thấp điểm, server chỉ cần xử lý 1 request/lần, còn những khung giờ cao điểm, server phải tăng batch size để phục vụ đầy đủ request từ người dùng, hoặc khi người dùng upload ảnh có độ phân giải khác nhau, hoặc nhập văn bản có độ dài không cố định. Tất cả những yếu tố này khiến việc cố định một kích thước duy nhất cho mô hình là không tối ưu.



Hình 20: Khai báo Dynamic Shape cho phép chạy engine với nhiều kích thước dữ liệu khác nhau.

Dynamic Shape là cơ chế xử lí các đầu vào có kích thước thay đổi. Cả ONNX Runtime và TensorRT đều hỗ trợ tính năng này nhưng với cơ chế hoàn toàn khác nhau.

IV.4.1. Dynamic Shape trong ONNX Runtime

ONNX Runtime hỗ trợ dynamic shape một cách tự nhiên và linh hoạt Khi nhận đầu vào mới, ORT kiểm tra kích thước các chiều. Nếu kích thước thay đổi so với lần chạy trước, hệ thống tự động cấp phát lại bộ nhớ và tính toán lại hình dạng các tensor trung gian trong thời gian thực thi. Ưu điểm: Người dùng gần như không cần cấu hình gì thêm. Chỉ cần xuất mô hình ONNX

với các chiều động, ORT sẽ tự động xử lí mọi kích thước đầu vào hợp lệ.

IV.4.2. Dynamic Shape trong TensorRT

Với TensorRT, việc hỗ trợ dynamic shape phức tạp hơn nhiều do mục tiêu cuối cùng là hiệu năng. Vì TensorRT biên dịch và đóng cứng các kernel tối ưu cho phần cứng cụ thể, nó cần biết trước kích thước dữ liệu để cấp phát bộ nhớ và chọn kernel tối ưu nhất. Nếu kích thước thay đổi liên tục, các kĩ thuật tối ưu hóa này có thể không còn đúng nữa.

Để thực hiện Dynamic Shape, TensorRT giới thiệu khái niệm **Optimization Profiles**. Khi build engine với dynamic shape, người dùng buộc phải khai báo trước một phạm vi kích thước thông qua 3 tham số:

- **Minimum (kMIN):** Kích thước nhỏ nhất mà engine hỗ trợ. TensorRT sẽ cấp phát bộ nhớ tối thiểu để chạy được kích thước này.
- **Optimal (kOPT):** Kích thước phổ biến nhất. TensorRT sẽ dành nhiều thời gian nhất để tìm ra kernel chạy nhanh nhất cho đúng kích thước này.
- **Maximum (kMAX):** Kích thước lớn nhất có thể xảy ra. TensorRT sẽ dựa vào đây để cấp phát trước bộ nhớ VRAM tối đa, đảm bảo không bị tràn bộ nhớ khi chạy thực tế.

Khi chạy thực tế, nếu dữ liệu đầu vào nằm trong khoảng $[kMIN, kMAX]$, engine sẽ chạy thành công. Nếu kích thước đầu vào trùng với $kOPT$, hiệu năng sẽ đạt tối ưu nhất. Nếu kích thước khác $kOPT$, TensorRT sẽ sử dụng các kernel tổng quát hơn, hiệu năng có thể thấp hơn một chút so với $kOPT$ nhưng vẫn đảm bảo tính chính xác.

V. Thực hành ONNX và TensorRT với YOLO26

V.1. Set up

Trong phần này, chúng ta sẽ thực hiện một mini-project sử dụng ONNX và TensorRT trên mô hình YOLO26, sau đó thực hiện đánh giá hiệu năng mô hình thông qua API.

Đầu tiên, ta khởi tạo môi trường Python bằng Conda.

```
!conda create -n onnx_tensorrt python=3.11.9 --y
!conda activate onnx_tensorrt
```

Clone Github Repository và cài đặt các thư viện cần thiết.

```
!git clone https://github.com/ThuanNaN/aio2025-onnx-tensorrt.git
!cd aio2025-onnx-tensorrt
!pip install -r requirements.txt
```

Tải dataset COCO val2014.

```
# COCO val2014
!wget http://images.cocodataset.org/zips/val2014.zip
!unzip val2014.zip -d ./images/
```

Tiếp theo, ta chạy file download_yolo.py để tải mô hình. Sau khi chạy file này, kết quả trả về tại thư mục chỉ định một file `yolo26l.pt` chứa trọng số của mô hình Yolo26.

download_yolo.py

```
from pathlib import Path
from ultralytics import YOLO

# Create models directory
MODELS_DIR = Path("models")
MODELS_DIR.mkdir(exist_ok=True)

def download_yolo26():
    """Download YOLO26 PyTorch model"""

    print("=" * 60)
    print("Downloading YOLO26 PyTorch Model")
```

```
    print("=" * 60)

    pytorch_path = MODELS_DIR / "yolo26l.pt"

    print("\nDownloading YOLO26 PyTorch model...")
    try:
        model = YOLO(str(pytorch_path))
        print(f"PyTorch model saved to: {pytorch_path}")
        size_mb = pytorch_path.stat().st_size / (1024 * 1024)
        print(f"  Size: {size_mb:.2f} MB")
    except Exception as e:
        print(f"Failed to download PyTorch model: {e}")
        return False

    print("\n" + "=" * 60)
    print("Download Complete!")
    print("=" * 60)
    return True


if __name__ == "__main__":
    download_yolo26()
```

V.2. Chuyển đổi mô hình YOLO26 sang ONNX và TensorRT

Để đơn giản hóa các quy trình chuyển đổi, thư viện Ultralytics cung cấp phương thức `model.export()` tích hợp sẵn ONNX và TensorRT. Khi được gọi, hàm này sẽ tự động hóa toàn bộ quy trình từ việc khởi tạo môi trường, truy vết mô hình, cho đến việc gọi trình biên dịch để tạo ra tệp tin đích cuối cùng.

Một số tham số trong `model.export()`:

- `format`: Chỉ định công cụ chuyển đổi. `format='onnx'` để xuất ra file `.onnx` hoặc `format='engine'` để xuất ra `.engine`
- `imgsz` và `batch`: Kích thước tensor được tự động sinh để thực hiện truy vết.
- `simplify`: Tối ưu hóa đồ thị tính toán.
- `nms`: Non-Maximum Suppression là tham số giúp loại bỏ các bounding-box trùng lặp trong bài toán Object Detection. `nms=False` thường được khuyến khích khi làm việc với TensorRT.
- `half`: Mô hình sẽ được chuyển đổi sang định dạng FP16
- `dynamic`: Sử dụng dynamic shape. Cần cấu hình thêm Optimization Profiles nếu `dynamic=True`.

V.2.1. YOLO26 to ONNX

File convert_2_onnx.py dùng để thực hiện chuyển đổi `yolo26.pt` sang định dạng `.onnx`. Ta bắt đầu phân tích nội dung trong file này:

Đầu tiên, ta import các thư viện cần thiết, đồng thời khai báo đường dẫn thư mục chứa mô hình YOLO.

convert_2_onnx.py

```python
from pathlib import Path
from ultralytics import YOLO
MODELS_DIR = Path("models")
```

Tiếp theo, viết hàm convert_to_onnx để chuyển đổi mô hình, sử dụng phương thức export() hỗ trợ sẵn của YOLO.

convert_2_onnx.py

```python
def convert_to_onnx():

    pytorch_path = MODELS_DIR / "yolo26l.pt"
    onnx_path = MODELS_DIR / "yolo26l.onnx"

    # Check if PyTorch model exists
    if not pytorch_path.exists():
        print(f"\n PyTorch model not found at: {pytorch_path}")
        print("  Please run download_yolo.py first")
        return False

    print("\nConverting to ONNX format...")
    try:
        model = YOLO(str(pytorch_path))
        # Export with static shapes for better GPU performance
        model.export(
            format="onnx",
            device=0,
            half=False,
            dynamic=False,
            simplify=True,
            batch=1,
            imgsz=640,
            nms=False
        )
        print(f"ONNX model saved to: {onnx_path}")
        size_mb = onnx_path.stat().st_size / (1024 * 1024)
        print(f"  Size: {size_mb:.2f} MB")
    except Exception as e:
        print(f"Failed to export to ONNX: {e}")
        return False
    print("Conversion Complete!")
    return True


if __name__ == "__main__":
    convert_to_onnx()
```

Sau khi chạy xong, một file `yolo26l.onnx` sẽ được lưu trong thư mục `models`. File này sẽ được dùng để thực thi trên các trình biên dịch.

V.2.2. YOLO26 to TensorRT

Để sử dụng TensorRT, ta tiến hành cài đặt thông qua terminal bằng cách chạy file `setup_tensorrt.sh`.

setup_tensorrt.sh

```
#!/bin/bash

os="ubuntu2404"
tag="10.14.1-cuda-13.0"
wget https://developer.download.nvidia.com/compute/tensorrt/10.14.1/local_installers/nv
                                                -tensorrt-local-repo-ubuntu2404-10.14.1-
                                                cuda-13.0_1.0-1_amd64.deb
sudo dpkg -i nv-tensorrt-local-repo-${os}-${tag}_1.0-1_amd64.deb
sudo cp /var/nv-tensorrt-local-repo-${os}-${tag}/*-keyring.gpg /usr/share/keyrings/
sudo apt-get update
sudo apt-get install tensorrt
```

Sau khi cài đặt xong, ta chạy file `convert_to_tensorrt.py` để chuyển mô hình gốc sang định dạng engine. Phân tích file này, đầu tiên khai báo các thư viện và đường dẫn mô hình.

convert_to_tensorrt.py

```
from pathlib import Path
from ultralytics import YOLO
MODELS_DIR = Path("models")
```

Viết hàm convert_to_tensorrt() để đổi mô hình sang engine. Cách chuyển đổi tương tự như ONNX, nhưng tham số đổi thành `format='engine'`.

convert_to_tensorrt.py

```
def convert_to_tensorrt():
    pytorch_path = MODELS_DIR / "yolo26l.pt"
    tensorrt_path = MODELS_DIR / "yolo26l.engine"

    # Check if PyTorch model exists
    if not pytorch_path.exists():
        print(f"\n PyTorch model not found at: {pytorch_path}")
        print("  Please run download_yolo.py first")
        return False
    print("\nConverting to TensorRT format...") #TensorRT requires NVIDIA GPU

    try:
        model = YOLO(str(pytorch_path))
        model.export(
            format="engine",
```

```
            device=0,
            half=False,
            dynamic=False,
            simplify=True,
            batch=1,
            imgsz=640,
            nms=False
        )
        print(f"TensorRT model saved to: {tensorrt_path}")
        size_mb = tensorrt_path.stat().st_size / (1024 * 1024)
    except Exception as e:
        print(f"Failed to export to TensorRT: {e}") #This is normal if TensorRT is not
                                                    installed or GPU is not available
        return False

    print("\n" + "=" * 60)
    print("Conversion Complete!")
    print("=" * 60)
    return True


if __name__ == "__main__":
    convert_to_tensorrt()
```

Cuối cùng, ta thu được file `yolo26l.engine` với cấu hình tương ứng với thiết bị NVIDIA của máy tính. File này không thể chạy trên thiết bị khác kiến trúc với phần cứng nó được xây nên.

V.3. Xây dựng ứng dụng API và Inference

Phần này cung cấp sẵn một ứng dụng client-server đơn giản. Cấu hình client sẽ thực hiện gửi các file mô hình đã chuẩn bị sẵn đến server, tại server sẽ thực hiện quá trình benchmark và trả kết quả về phía client. Để inference hai mô hình ONNX và TensorRT đã chuẩn bị ở phần trước, ta cũng sử dụng hàm hỗ trợ sẵn có của YOLO.

V.3.1. Load các file mô hình đã chuẩn bị

Trong `app/utils/load_model.py`, class ModelLoader sử dụng `YOLO(model_path, task='detect')` tự động nhận diện đuôi file .onnx hay .engine để cấu hình backend phù hợp.

Khởi tạo đối tượng ModelLoader:

app/utils/load_model.py

```
class ModelLoader:
    def __init__(self, model_paths: Optional[Dict[str, str]] = None):
        self.model_paths = model_paths or {
            "pytorch": "models/yolo26l.pt",
            "onnx": "models/yolo26l.onnx",
            "tensorrt": "models/yolo26l.engine"
```

```
        }
        self._models: Dict[str, Any] = {
            "pytorch": None,
            "onnx": None,
            "tensorrt": None
        }
```

Bên trong ModelLoader, khởi tạo phương thức load_model(). Phương thức này nhận đầu vào là định dạng file mô hình và đường dẫn và gọi đến các phương thức load mô hình cụ thể.

app/utils/load_model.py

```
#class ModelLoader:
#   ...
    def load_model(self, model_type: str, model_path: Optional[str] = None):
            path = model_path if model_path else self.model_paths.get(model_type)

            if not path:
                raise ValueError(f"No path configured for model type: {model_type}")

            if model_type == "pytorch":
                return self._load_pytorch_model(path)
            elif model_type == "onnx":
                return self._load_onnx_model(path)
            elif model_type == "tensorrt":
                return self._load_tensorrt_model(path)
            else:
                raise ValueError(f"Unsupported model type: {model_type}")
```

Tiếp theo, tạo thêm các phương thức load mô hình, các phương thức này trả ra cấu hình backend cụ thể tương tứng với định dạng đầu vào.

app/utils/load_model.py

```
#class ModelLoader:
#   ...
    def _load_pytorch_model(self, model_path: str):
        """Load PyTorch YOLO model"""
        if self._models["pytorch"] is None:
            self._models["pytorch"] = YOLO(model_path, task='detect')
        return self._models["pytorch"]

    def _load_onnx_model(self, model_path: str):
        """Load ONNX model with GPU acceleration"""
        if self._models["onnx"] is None:
            self._models["onnx"] = YOLO(model_path, task='detect')
        return self._models["onnx"]


    def _load_tensorrt_model(self, model_path: str):
```

```
        """Load TensorRT model"""
        if self._models["tensorrt"] is None:
            self._models["tensorrt"] = YOLO(model_path, task='detect')
        return self._models["tensorrt"]
```

V.3.2. Inference

Inference sẽ được thực hiện trong `app/api/endpoints/yolo.py`. Hàm predict sẽ gọi phương thức model.predict() của đối tượng YOLO để thực hiện inference.

Bắt đầu khai báo hàm predict() để inference dữ liệu từ tập COCO và trả về dự đoán. Đầu tiên hàm này nhận đầu vào là các đường dẫn, kích thước ảnh và các thông số của mô hình YOLO. Sau đó thực hiện tiền xử lí dữ liệu.

app/api/endpoints/yolo.py

```
router = APIRouter()
model_loader = ModelLoader()

@router.post("/predict")
async def predict(
    file: UploadFile = File(...),
    model_format: Literal["pytorch", "onnx", "tensorrt"] = Form("pytorch"),
    conf_threshold: float = Form(0.25),
    iou_threshold: float = Form(0.45),
    img_size: int = Form(640)
):
    try:
        # Read and validate image
        image_bytes = await file.read()
        image = Image.open(io.BytesIO(image_bytes))

        # Get model path
        model_path = model_loader.model_paths.get(model_format, "")
        if not model_path:
            raise HTTPException(status_code=400, detail=f"Invalid model format: {
                                                model_format}")

        # Preprocess image
        processed_image, original_size = preprocess_image(image, img_size)

        # Perform inference based on format
        start_time = time.time()
#...)
```

Tiếp theo, tiến hành load các mô hình với đường dẫn đã cung cấp. Rồi sử dụng phương thức predict() của model để inference ra kết quả dự đoán.

app/api/endpoints/yolo.py

```
#async def predict( ...

        if model_format == "pytorch":
            model = model_loader.load_model("pytorch")
            results = model.predict(
                processed_image,
                conf=conf_threshold,
                iou=iou_threshold,
                verbose=False
            )
            detections = postprocess_results(results, conf_threshold)

        elif model_format == "onnx":
            model = model_loader.load_model("onnx")
            results = model.predict(
                processed_image,
                conf=conf_threshold,
                iou=iou_threshold,
                verbose=False
            )
            detections = postprocess_results(results, conf_threshold)
        elif model_format == "tensorrt":
            model = model_loader.load_model("tensorrt")
            results = model.predict(
                processed_image,
                conf=conf_threshold,
                iou=iou_threshold,
                verbose=False
            )
            detections = postprocess_results(results, conf_threshold)
#...)
```

Cuối cùng, đo thời gian thực thi và trả kết quả dự đoán ra định dạng json.

app/api/endpoints/yolo.py

```
        inference_time = time.time() - start_time

        return JSONResponse(content={
            "success": True,
            "model_format": model_format,
            "inference_time": round(inference_time, 4),
            "image_size": {
                "original": {"width": original_size[0], "height": original_size[1]},
                "processed": {"width": img_size, "height": img_size}
            },
            "detections_count": len(detections),
            "detections": detections,
            "parameters": {
                "conf_threshold": conf_threshold,
```

```
                "iou_threshold": iou_threshold
            }
        })

    except FileNotFoundError:
        raise HTTPException(
            status_code=404,
            detail=f"Model file not found: {model_path}. Please ensure the model is
                                                downloaded."
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Inference error: {str(e)}")
```

Tất cả các bước trên sẽ được thực hiện ở phía Server-Side. Bây giờ, chúng ta chạy file `benchmark.py`, file này thực hiện so sánh các mô hình YOLO26 Pytoch, ONNX và TensorRT thông qua việc gửi API request đến server để tính toán các thông số, sau đó tạo báo cáo cuối cùng. Output của file này bao gồm các chỉ số:

- Mean inference time with standard deviation
- Throughput (FPS)
- Min/Max/Median inference times
- P95/P99 percentiles
- Average detections per image
- Performance comparison across model formats
- File JSON lưu kết quả và biểu đồ tại benchmark_results

Lưu ý đo đạc các chỉ số này, ta cần chạy trước một vài mẫu dữ liệu để **warm-up**. Lí do là ở những lần chạy đầu tiên, hệ thống cần thời gian để khởi động vi xử lí và load các kernel tính toán phù hợp. Việc đo các chỉ số tại giai đoạn này làm kết quả thu được thường không chính xác với thực tế.

Bây giờ, ta thực hiện chạy server, sau đó chạy `benchmark.py` từ phía client.

```
#Terminal 1
!run_app.sh

#Ternimal 2
!python benchmark.py

#Advanced Options
# Test mô hình với số lượng ảnh cụ thể
python benchmark.py --max-images 50

# Test với file ảnh cụ thể.
```

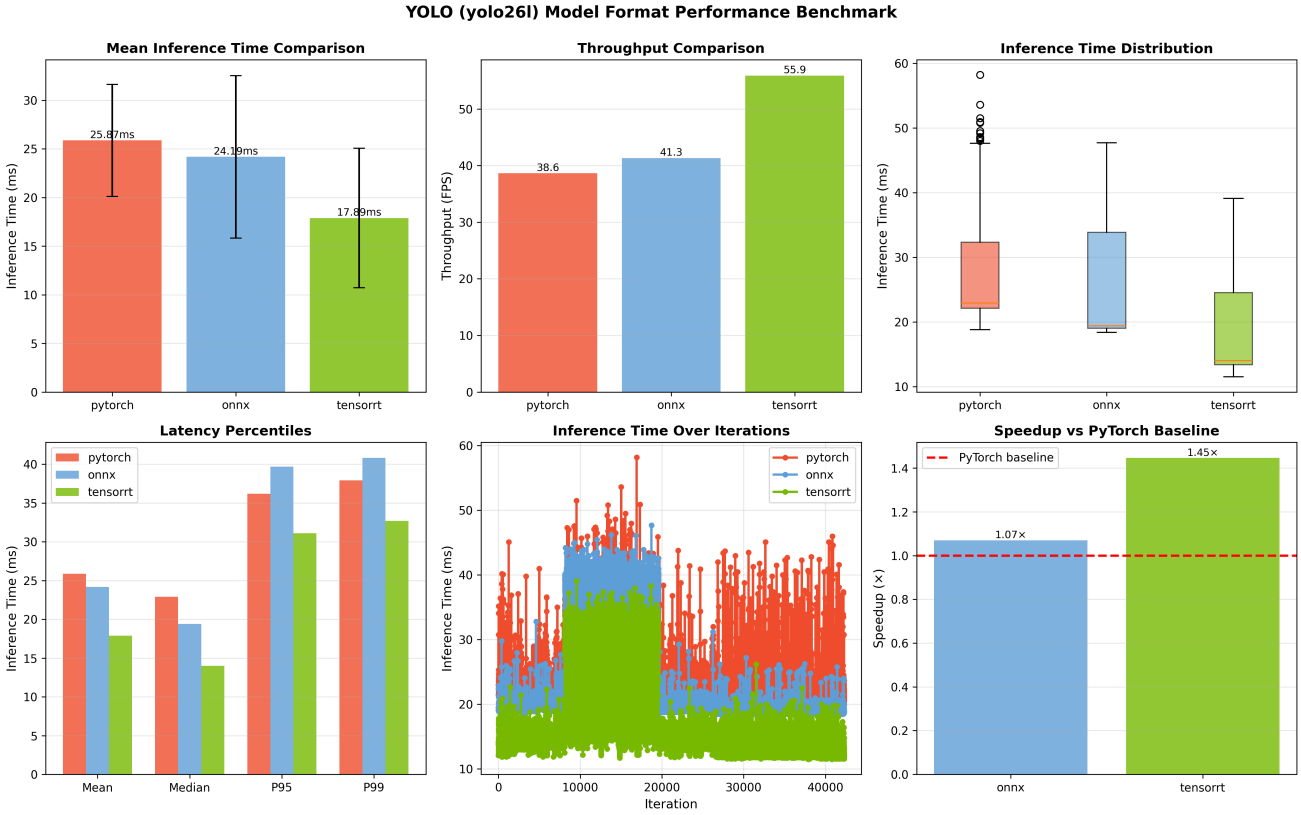
```
!python benchmark.py --image images/val2014/COCO_val2014_000000000042.jpg

# Test với mô hình tùy chọn
!python benchmark.py --models onnx tensorrt --max-images 20

# Custom số lượng vòng lặp và số mẫu warm-up.
!python benchmark.py --iterations 20 --warmup 5

# Chỉ định API server URL
!python benchmark.py --url http://localhost:8001
```

Kết quả cuối cùng cũng trả về các bảng so sánh hiệu năng giữa các trình biên dịch trên GPU NVIDIA như hình bên dưới:



Hình 21: Các biểu đồ so sánh hiệu năng của mô hình YOLO26 triển khai trên Pytorch, ONNX và TensorRT

Dựa vào đồ thị, có thể nhận xét rằng mô hình YOLO26 triển khai trên TensorRT đạng hiệu năng cao nhất cả về băng thông lẫn tốc độ thực thi do đặc điểm được tối ưu trên thiết bị NVIDIA. Ngay sau đó là ONNX và cuối cùng là trình chạy mặc định của Pytorch.

VI. Câu hỏi ôn tập

Câu 1: Giả sử một mô hình được viết bằng Pytorch có chứa điều kiện:

```
if confidence > 0.5:
    return box_A
else:
    return box_B
```

Mô hình sau đó được xuất sang ONNX bằng torch.onnx.export với Dummy_Input có độ tin cậy bằng 0.8. Khi triển khai, bạn phát hiện ra rằng dù input thực tế có độ tin cậy thấp (0.1), mô hình ONNX vẫn luôn trả về box_A. Tại sao?

A) Do ONNX Runtime bị lỗi bộ nhớ đệm.

B) Do chế độ Tracing của hàm export không xử lí được đầy đủ trường hợp.

C) Do giảm độ chính xác dữ liệu gây ra sai lệnh giá trị.

D) Do Python bỏ qua các lệnh rẽ nhánh khi biên dịch sang ONNX.

Câu 2: Khi triển khai mô hình ResNet50 bằng ONNX Runtime trên GPU. Thời gian tính toán chỉ mất 2ms nhưng tổng thời gian trả về kết quả lên tới 10ms. Sử dụng chức năng Profiling để debug cho thấy GPU phải chờ CPU gửi dữ liệu rất lâu. Sử dụng tính năng nào của ONNX Runtime để giảm độ trễ này?

A) GraphOptimizationLevel.ORT_ENABLE_ALL

B) IO Binding

C) Tăng intra_op_num_threads

D) Chuyển sang dùng TensorRTExecutionProvider.

Câu 3: Export một mô hình Transformer mới nhất từ PyTorch sang ONNX để chạy trên một thiết bị biên cũ đã cài sẵn ONNX Runtime từ 2 năm trước. Khi chạy, hệ thống báo lỗi "Unknown Operator". Nguyên nhân và cách xử lí hợp lí nhất?

A) Mô hình quá lớn, cần cắt bớt các layer dư thừa.

B) Phiên bản opset_version khi export cao hơn mức mà Runtime cũ hỗ trợ. Cần export lại với opset_version thấp hơn

C) Phiên bản opset_version khi export thấp hơn mức mà Runtime cũ hỗ trợ. Cần export lại với opset_version cao hơn

D) File ONNX bị lỗi trong quá trình chuyển đổi.

Câu 4: Export mô hình bằng ONNX đã cài đặt dynamic_axes cho batch_size. Tuy nhiên, khi chạy với batch_size = 8, ONNX Runtime báo lỗi:

```
#...
Error: Non-zero status code returned while running Reshape node. Name:'Reshape_123'
```

Kiểm tra thì thấy node này cố gắng reshape tensor thành kích thước cố định dựa trên batch_size = 1. Nguyên nhân có thể là gì?

A) Mô hình sử dụng phép toán view() hoặc reshape() với một kích thước cứng.

B) ONNX không hỗ trợ batch size lớn hơn 1.

C) File ONNX bị hỏng trong quá trình build.

D) Do chưa bật tính năng Dynamic Shape trên Runtime.

Câu 5: export một mô hình Ngôn ngữ lớn (LLM) khoảng 7 tỷ tham số có file .pt nặng 14GB sang định dạng ONNX. Tuy nhiên, quá trình export bị lỗi với thông báo liên quan đến "Protocol Buffers limit". Nguyên nhân và giải pháp chuẩn cho vấn đề này là gì?

A) ONNX không hỗ trợ mô hình lớn hơn 2GB, phải cắt nhỏ mô hình thành nhiều file .onnx rồi chạy tuần tự.

B) RAM của máy không đủ. Cần mua thêm RAM.

C) Định dạng Protobuf của ONNX có giới hạn cứng kích thước file là 2GB. Cần đặt tham số save_as_external_data=True để lưu trọng số ra các file nhị phân rời bên ngoài file .onnx.

D) Phải chuyển sang dùng TensorRT vì ONNX đã lỗi thời.

Câu 6: Tiến hành build một Docker Image chứa ứng dụng AI. Trong Dockerfile, thực hiện lệnh convert mô hình ONNX sang TensorRT Engine (.engine) để tiết kiệm thời gian khởi động cho user. Sau đó build image này trên máy Server có GPU Tesla T4. Khi khách hàng kéo image về chạy trên máy có GPU RTX 3090, ứng dụng báo lỗi không load được engine. Tại sao?

A) Do RTX 3090 yếu hơn Tesla T4.

B) Do Docker không hỗ trợ GPU.

C) Do thiếu thư viện PyTorch trong Docker.

D) Do TensorRT Engine bị gắn chặt với kiến trúc phần cứng.

Câu 7: Kĩ thuật Layer Fusion của TensorRT giúp tăng tốc độ suy luận chủ yếu nhờ vào cơ chế nào?

A) A.Giảm số lượng tham số cần lưu trữ trong VRAM.

B) Giảm bớt số lần truy cập bộ nhớ bằng cách tính toán liên tục nhiều phép toán trong một lần đọc dữ liệu.

C) Chuyển đổi toàn bộ các phép nhân ma trận thành phép cộng để tính nhanh hơn.

D) ự động ép xung (overclock) GPU khi chạy các lớp mạng phức tạp.

Câu 8: Khi cấu hình Dynamic Shapes cho TensorRT, nếu đặt optimal shape là (1, 3, 224, 224) nhưng max shape là (32, 3, 1024, 1024). Hậu quả về mặt tài nguyên khi load engine này là gì?

A) A. TensorRT chỉ cấp phát bộ nhớ RAM vừa đủ cho optimal shape, giúp tiết kiệm tài nguyên.

B) TensorRT buộc phải cấp phát trước lượng bộ nhớ đủ để chứa trường hợp lớn nhất, dẫn đến lãng phí lớn nếu thực tế chỉ chạy ở optimal shape.

C) Engine sẽ load rất nhanh nhưng chạy chậm ở các kích thước lớn hơn optimal shape.

D) Engine sẽ tự động crash nếu batch size lớn hơn 1.

Câu 9: Tiến hành dùng trtexec để build engine cho một mô hình 3D-UNet rất lớn. Quá trình build thất bại với lỗi liên quan đến bộ nhớ, dù VRAM còn trống rất nhiều. Log báo lỗi 'Try increasing the workspace size'. Tham số workspace, hoặc memory_pool_limit trong bản mới, thực chất dùng để làm gì trong quá trình Build?

A) Chứa trọng số của mô hình.

B) Là vùng nhớ tạm để TensorRT thử nghiệm các kernel khác nhau.

C) Để lưu trữ giá trị input/output

D) Để lưu trữ toàn bộ giá trị tính toán trung gian của mô hình.

Câu 10: Một đội ngũ AI vừa huấn luyện xong một phiên bản YOLO mới có độ chính xác vượt trội. Lúc này, ta cần đưa phiên bản này vào Model Registry (ví dụ: MLflow, DVC, hoặc Weights & Biases) để chuẩn bị deploy xuống hàng nghìn xe ô tô sử dụng GPU Jetson Orin và server giả lập sử dụng GPU A100. Để đảm bảo tính tái sử dụng, tương thích đa phần cứng và khả năng rollback, nên lưu trữ tệp tin nào vào Model Registry?

A) Lưu trực tiếp file TensorRT Engine (.engine hoặc .plan) đã được build sẵn cho Jetson Orin. Khi deploy chỉ việc tải xuống và chạy, giúp giảm thời gian khởi động tối đa.

B) Chỉ lưu file trọng số gốc PyTorch (.pt) cùng với source code huấn luyện. Khi deploy, các thiết bị sẽ tự chạy script export ra ONNX rồi build ra Engine.

C) Lưu file ONNX (.onnx) đã được export và tối ưu hóa, kèm theo file cấu hình tham số build TensorRT. Không lưu file .engine.

D) Lưu Docker Image chứa sẵn môi trường và file .engine bên trong.

VII. Tài liệu tham khảo

[1] ONNX Community, *Onnx documentation*, https://onnx.ai/onnx/index.html, Accessed: 2026-02-08, 2025.

[2] T.-T. Wang et al., *Export a pytorch model to onnx*, https://docs.pytorch.org/tutorials/beginner/onnx/export_simple_model_to_onnx_tutorial.html, PyTorch Tutorials, accessed February 08, 2026, 2025.

[3] ONNX Community, *Tensorflow-onnx: Convert tensorflow, keras, tensorflow.js and tflite models to onnx*, https://github.com/onnx/tensorflow-onnx, GitHub repository, accessed February 08, 2026, 2024.

[4] ONNX Community, *Sklearn-onnx: Convert scikit-learn models to onnx*, https://onnx.ai/sklearn-onnx/, Documentation, accessed February 08, 2026, 2025.

[5] ONNX Community, *Onnxmltools: Tools for converting models to onnx*, https://github.com/onnx/onnxmltools, GitHub repository, accessed February 08, 2026, 2024.

[6] ONNX Runtime developers, *Onnx runtime*, https://onnxruntime.ai/, Accessed: 2026-02-08, 2021.

[7] NVIDIA, *Tensorrt*, https://developer.nvidia.com/tensorrt, NVIDIA Developer documentation, accessed February 08, 2026, 2025.